Dynamic Monitoring of Radiological Pollution through Deep Learning

Anomaly classification in noisy gamma-dose time series

Pedro

CentraleSupélec — Pôle Projet
Supervised by Elisabeth Lahalle (L2S, CentraleSupélec)

May 2026

Abstract

Continuous monitoring of radiological pollution relies on environmental surveillance beacons that record the rate of gamma photons reaching a detector. The recorded signal is heavily contaminated by sensor noise and slow baseline drift, which makes the early identification of anomalous radiological events — spikes, plateaus, ramp-ups, ramp-downs — a non-trivial detection *and* classification problem.

This report documents the work carried out during the *Pôle Projet* at CentraleSupélec on this subject. The contribution is two-fold. First, a *synthetic dataset generator* is built that calibrates a stochastic background process on real 2015 gamma dose recordings and injects six hand-designed, mathematically parameterised anomaly templates with controlled amplitude, duration and shape deformation. The generator produces a labelled time series of ten million points, large enough to train modern neural networks. Second, three neural network architectures of increasing complexity are designed, trained and compared on the classification task: a compact **1D-CNN** (23 k parameters), a **1D residual network** (75 k parameters) and a **hybrid CNN+Transformer model with multi-head self-attention** (110 k parameters).

The three models are evaluated at the level of individual events, not individual samples, which better matches operational use. On a held-out synthetic test segment containing 59 events, all three models achieve strong performance, with the residual network reaching 100% event-level macro F1, the attention model reaching 95.5%, and the baseline 1D-CNN reaching 93.4%. A dedicated robustness study reveals that residual connections and self-attention substantially improve graceful degradation when the background noise or the anomaly shape moves outside the training distribution. Finally, the models are run on unlabelled real 2015 data: this stress-test exposes a clear *distribution shift* between the densely-injected synthetic training set and the sparse, almost purely-noise real signal, and we analyse the consequences — including a degenerate collapse of the attention model to one class — with concrete recommendations for the next iteration of the project.

Keywords: time series classification, 1D convolutional neural networks, residual networks, self-attention, transformer encoder, radiological monitoring, synthetic data generation, distribution shift.

Contents

List of Figures

Chapter 1

Introduction

1.1 Context: continuous radiological surveillance

Environmental monitoring beacons measure the local radiation level by counting the gamma photons reaching a scintillation detector in a fixed sampling interval — typically one minute. The resulting time series, called the *gamma dose*, reflects the natural radioactive background of the environment, modulated by atmospheric conditions (temperature, humidity, pressure) and, hopefully rarely, by accidental or deliberate radiological events such as the dispersion of a radioactive plume, a leak from a nearby facility, or the passage of a contaminated source.

The operational goal of a network of such beacons is to detect and classify those rare events *as early as possible*, so that mitigation decisions can be taken with confidence. Three properties make this task fundamentally harder than industrial anomaly detection in mechanical or financial time series:

1. **High noise-to-signal ratio.** The Poisson-like sensor statistics produce a high-frequency dispersion that is comparable in magnitude to many anomalies of interest.
2. **Slow baseline drift.** The mean radiological background is not stationary — it slowly wanders with weather and seasonal factors, so an anomaly cannot be defined by a fixed threshold on the raw signal.
3. **Class diversity.** Different physical phenomena leave different temporal signatures: a sudden spike (a passing source), a slow ramp-up (a slowly approaching plume), a sustained plateau (a deposited contamination), and so on. A meaningful response requires not just detection but classification.

1.2 Problem statement

The project, supervised by Élisabeth Lahalle at the *Laboratoire des Signaux et Systèmes* (L2S, CentraleSupélec), formulates the problem as a **streaming multi-class classification task** on a univariate gamma-dose time series. Given a sliding window of recent samples, the system must output one of seven labels:

`Normal Background` | `bell` | `bell_sq` | `fast_ascend` | `fast_descend` | `M` | `square`.

The first six anomaly classes correspond to canonical shapes identified together with the supervisor as representative of the radiological phenomena one might encounter on a beacon. Their precise mathematical parameterisation is given in chapter 3.

1.3 Why deep learning?

A classical signal-processing pipeline — baseline removal followed by a hand-tuned shape-matching template bank — would in principle suffice on clean shapes, but it would scale poorly to:

- *Shape variability*: real events do not match a template perfectly; their amplitude, duration and time-warping vary.
- *Noise*: at the noise levels observed in the real data, cross-correlation with a template becomes unreliable.
- *Class proliferation*: adding a new event class requires redesigning the bank.

Deep neural networks are attractive because they learn *their own* features from data, and because the convolutional and attention building blocks of modern architectures are very effective at recognising patterns in 1D signals. The reference project brief[1] explicitly suggests evaluating convolutional and attention layers, and we follow that guidance.

1.4 Contributions

Concretely, this project produces:

1. A **statistical characterisation** of four months of real 2015 gamma dose recordings, from which we extract a usable noise model (Gaussian high-frequency component and Ornstein–Uhlenbeck baseline drift, section 3.1).
2. A **synthetic data generator** that injects six parameterised anomaly templates with controlled amplitude, duration and shape deformation into a ten-million-point simulated signal (chapter 3).
3. Three **PyTorch architectures** of increasing complexity — a baseline 1D-CNN, a residual variant, and a hybrid CNN+Transformer model with multi-head self-attention — all sharing the same I/O contract (chapter 4).
4. A **training and event-level evaluation pipeline** that bridges the gap between sample-wise classification accuracy and operational event-detection metrics (chapters 4 and 5).
5. A dedicated **robustness study** that sweeps background noise and shape deformation independently and reports the F1-degradation curves of all three architectures (chapter 6).
6. Application of the trained models to real, unlabelled 2015 data, together with a careful **diagnostic analysis** of where and why they fail in the real world, leading to concrete recommendations (chapter 7).

1.5 Report organisation

The report is laid out in six parts. chapter 2 provides the theoretical background — convolutions, residual learning, attention mechanisms and the relevant training machinery — with enough mathematical detail to make the architectures self-contained for a reader without a deep-learning background. chapter 3 describes the real data and the synthetic generator built from it. chapter 4 present the three neural networks and how they are trained. chapters 5 and 6 report the quantitative results, first on a clean test segment then under controlled stress. chapter 7 discusses

[1] See `biblio/projet.pdf`.

the qualitative behaviour on real recordings and the open questions that remain. chapter 8 concludes and lists the next steps that are on the table for the project's next iteration.

All code, data generation scripts and figure-generation utilities necessary to reproduce the contents of this report are included in the project repository.

Chapter 2

Theoretical background

This chapter gathers the mathematical machinery that the three architectures of chapter 4 rest on. The goal is to make the report *self-contained*: a reader without prior exposure to deep learning should, after this chapter, be able to read off the operations performed inside each of our three networks and understand why they were chosen.

We start with the abstract framing of the task as a windowed classification problem (section 2.1). We then describe in detail the three building blocks that compose our architectures: convolutional layers and the supporting normalisations (section 2.2), residual connections (section 2.3), and the multi-head self-attention mechanism that powers Transformer encoders (section 2.4). Finally, we summarise the training ingredients that all three networks share (section 2.5).

2.1 Framing: time-series classification by windowing

Let $x \in \mathbb{R}^N$ denote the gamma-dose signal sampled at uniform intervals (one minute, in our case). We want to assign each sample x_t to one of $K = 7$ classes (six anomaly types plus the normal background).

Windowing. Operating on a single sample is hopeless: the classes are defined by the temporal *shape* of an event, not by the instantaneous value. We therefore process the signal through a sliding window of length W samples:

$$\mathbf{x}^{(t)} = (x_{t-W+1}, x_{t-W+2}, \dots, x_t) \in \mathbb{R}^W, \tag{2.1}$$

and we ask each network to output, from the window $\mathbf{x}^{(t)}$, a probability distribution $\hat{p}^{(t)} \in \Delta^{K-1}$ over the K classes[1]. In our experiments $W = 512$ samples (about 8.5 hours at 1 sample / minute).

Successive windows overlap (consecutive predictions share most of their input), which lets us *accumulate* probabilities at inference time into a continuous, sample-level confidence trace (section 5.3). At training time, windows are extracted at a larger stride (32 samples) to reduce redundancy.

Labelling rule. Because a single window may contain a mixture of background and event samples, we follow a simple rule: the window inherits the most frequent non-zero label inside it if more than 10% of its samples are anomalous, and is labelled `Normal Background` otherwise

[1]Here Δ^{K-1} denotes the standard $(K-1)$-simplex, i.e. the set of non-negative vectors that sum to 1.

(fig. 2.1). This makes the boundaries of events slightly fuzzy — a deliberate choice that helps the network generalise to events whose exact start and end are uncertain.

Figure 2.1. Sliding-window construction on a fragment of the synthetic signal. Each coloured bar represents one 512-sample training window; red bars indicate windows that contain more than 10% anomaly points, so they inherit the underlying class label.

Mathematical objective. Letting $f_\theta : \mathbb{R}^W \to \mathbb{R}^K$ denote the network with parameters θ, the empirical objective is the (weighted) cross-entropy

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x},y)\in\mathcal{D}} w_y \log \operatorname{softmax}(f_\theta(\mathbf{x}))_y, \tag{2.2}$$

where $\mathcal{D}$ is the training set of (window, label) pairs, $y \in \{0, \dots, K-1\}$ the integer label, and $w_y > 0$ a class weight that compensates for class imbalance, defined in section 2.5.2.

2.2 Convolutional layers and supporting normalisations

The cornerstone of all three architectures is the **one-dimensional convolution**.

2.2.1 The 1D convolution operator

Let $\mathbf{u} \in \mathbb{R}^{C_{\text{in}} \times L}$ denote an input sequence with C_{in} channels and length L, and let $\mathbf{w} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k}$ denote a learnable kernel of C_{out} output channels with support of size k. The convolution output $\mathbf{v} \in \mathbb{R}^{C_{\text{out}} \times L'}$ has entries:

$$\mathbf{v}_{c,t} = \sum_{c'=1}^{C_{\text{in}}} \sum_{i=1}^{k} \mathbf{w}_{c,c',i} \cdot \mathbf{u}_{c',\, s\cdot t+i-p} + b_c, \tag{2.3}$$

where b_c is a learnable bias, s is the *stride* and p is the *padding*. The output length is $L' = \lfloor (L + 2p - k)/s \rfloor + 1$. Three observations matter:

- The kernel $\mathbf{w}_{c,\cdot,\cdot}$ is shared across all time positions t, so the operator is *translation equivariant*: shifting the input shifts the output by the same amount. This is exactly what we want for an anomaly that may occur anywhere in the window.
- The number of parameters is $C_{\text{out}} \cdot C_{\text{in}} \cdot k$ (plus C_{out} biases), *independent of* L. Convolutions are therefore much cheaper than the dense fully-connected alternative, which would have $C_{\text{out}} \cdot C_{\text{in}} \cdot L \cdot L'$ parameters.
- Each output channel c acts as a learned *feature detector*: it lights up where the input matches a particular local pattern of length k.

Receptive field. Stacking several convolutional layers, optionally with sub-sampling (stride or pooling, see below), the *receptive field* of a deeper neuron — the range of input samples that can affect its value — grows roughly linearly with depth and exponentially with the sub-sampling factor. Our $W = 512$ window is short enough that three stride-2 pooling steps (giving a receptive field of ~ 50 samples on top of $k = 7$ kernels) already cover the duration of the smallest expected anomalies (period ≥ 200 samples once temporal pooling is accounted for).

2.2.2 Pointwise nonlinearity: ReLU

Stacking purely linear convolutions yields, by composition, another linear operator and so cannot learn rich features. Between successive convolutions we therefore apply an element-wise non-linearity. We use the *rectified linear unit*, $\mathrm{ReLU}(x) = \max(0, x)$, which is cheap to compute, has a non-vanishing gradient on its positive half-line, and has become the de facto standard activation in convolutional networks. The two normalisation variants used below already absorb the negative-half collapse problem ("dying ReLU"), so we do not need a fancier activation.

2.2.3 Pooling

After a few convolutions we typically reduce the temporal resolution by *pooling* a contiguous group of values into one. Our networks use two kinds of pooling.

Max-pooling. Of the values in a window of size k_p, keep only the maximum: $\mathrm{MaxPool}_k(u)_t = \max_{i \in [1, k_p]} u_{s_p t + i}$, with stride $s_p = k_p$ in our case so that successive max-pools do not overlap. The output is shorter by a factor k_p. Max-pooling makes the representation locally invariant to small temporal shifts of an anomaly.

Adaptive average pooling. Used near the end of the network to bring the temporal axis down to a fixed length regardless of the input window size: it averages the input into a target number of bins. Our networks emit 4 temporal bins from `AdaptiveAvgPool1d(4)`, which preserves a coarse temporal shape ("did the activation peak in the first or the last quarter of the window?") while removing the exact location, before the classifier head.

2.2.4 Batch normalisation

A common pathology of deep networks is that the distribution of activations inside the network shifts during training, slowing down convergence (the so-called *internal covariate shift*). Batch normalisation [5] addresses this by standardising each channel of each layer over the current mini-batch:

$$\mathrm{BN}(\mathbf{u})_{c,t} = \gamma_c \cdot \frac{\mathbf{u}_{c,t} - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}} + \beta_c, \tag{2.4}$$

where μ_c, σ_c^2 are the channel mean and variance computed over the batch dimension (and the time dimension for 1D-BN), and γ_c, β_c are learnable rescaling parameters.

At inference time, μ_c and σ_c^2 are replaced by exponential moving averages collected during training. This means BN behaves *differently* at training and inference time, which can become problematic when there is a distribution shift between training and deployment — a subtlety that motivates the GroupNorm variant in our two more advanced architectures.

2.2.5 Group normalisation

Group normalisation [9] performs the same affine standardisation *but* computes the statistics μ, σ^2 over a partition of the channels into G groups, independently for each sample. There are

no running statistics to maintain; train and inference behaviour are identical, and the operator becomes amplitude-invariant in a window-by-window fashion. This robustness against shifts in absolute scale is exactly the property we need in a stem that processes signals with a fluctuating baseline. Our ResNet 1D and CNN+Attention models use GroupNorm with 8 groups (or fewer for narrow channel widths) throughout their convolutional stems.

2.3 Residual connections

The vanishing-gradient problem. A pure stack of convolutional layers $f_1 \circ f_2 \circ \cdots \circ f_L$ becomes harder to optimise as L grows: the gradient of the loss with respect to early-layer parameters is a product of L Jacobian factors, and unless each of them is exactly orthogonal it tends to either vanish or explode exponentially.

Residual blocks. He et al. [3] proposed to reformulate a stack of two or three layers as a learnable *residual* to the identity:

$$\mathrm{ResBlock}(\mathbf{u}) = \mathrm{ReLU}\big(\,\mathbf{u} + \mathcal{F}(\mathbf{u})\,\big), \tag{2.5}$$

where $\mathcal{F}$ is a small sub-network (here: Conv1D $\to$ GN $\to$ ReLU $\to$ Conv1D $\to$ GN). The shortcut $+\mathbf{u}$ guarantees that the block can degenerate to the identity ($\mathcal{F} = 0$) at no cost, which means adding a residual block to a working network cannot make things worse. In gradient terms the backward pass propagates *additively* through the shortcut, so deep stacks remain trainable.

When the input and the output of $\mathcal{F}$ have different channel counts, the shortcut is replaced by a 1×1 convolution that projects them into compatible spaces. We use this only at block transitions, where the channel width grows from $16 \to 32 \to 64$.

Why residual learning helps anomaly classification. For our problem the residual block can be intuitively read as "copy the input through, then *add* a learnt correction": in early blocks the correction extracts local shape primitives (peaks, ramps); in later blocks it refines them by re-using the original signal context that has been preserved by the shortcut.

2.4 Attention and the Transformer encoder

The third architecture in this report augments a small convolutional stem with a *Transformer encoder* [8]. Transformers are now the dominant architecture in natural language and sequence modelling; they have also proved very competitive on time-series classification [4]. The mechanism that powers them is *self-attention*, which we now describe carefully.

2.4.1 Self-attention

The input to a self-attention layer is a sequence of T token vectors $\mathbf{Z} \in \mathbb{R}^{T\times d}$ (one vector per time step, of dimension d). The layer first projects $\mathbf{Z}$ into three learnable representations called *queries*, *keys* and *values*:

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}^{Q}, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}^{K}, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}^{V}, \tag{2.6}$$

where $\mathbf{W}^{Q}, \mathbf{W}^{K}, \mathbf{W}^{V} \in \mathbb{R}^{d\times d_h}$ are learnable parameter matrices (d_h is the per-head dimension). The attention output is then

$$\mathrm{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \mathrm{softmax}\!\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_h}}\right)\mathbf{V}. \tag{2.7}$$

The matrix $\mathbf{A} = \mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d_h}) \in \mathbb{R}^{T\times T}$ has rows that sum to 1; its entry A_{ij} is the fraction of token i's output that comes from token j. The output sequence at row i is therefore a *convex combination of all input value vectors*, weighted by how compatible the query of token i is with the key of each other token. The scaling by $\sqrt{d_h}$ keeps the dot products from saturating the softmax in high dimensions.

Why this matters for our problem. Convolutional layers have a bounded receptive field: a neuron in layer ℓ depends only on samples within a window of size proportional to ℓ. Self-attention, by contrast, lets every output position attend to *every* input position in a single layer, with a *learnt*, data-dependent weighting. This long-range, content-based mixing is exactly what is needed to disambiguate, say, a sharp rise that could be a `fast_ascend` or just a noise excursion, by looking at what happens *after* the rise within the same window.

2.4.2 Multi-head attention

A single attention head can only learn one notion of similarity. To let the model attend to several relations in parallel, multi-head attention runs h independent attention heads on parallel projections of the same input, and concatenates their outputs:

$$\mathrm{MultiHead}(\mathbf{Z}) = \mathrm{Concat}(\mathbf{O}_1, \dots, \mathbf{O}_h)\mathbf{W}^O, \qquad \mathbf{O}_j = \mathrm{Attention}(\mathbf{Z}\mathbf{W}_j^Q, \mathbf{Z}\mathbf{W}_j^K, \mathbf{Z}\mathbf{W}_j^V). \quad (2.8)$$

We use $h = 4$ heads with $d_h = d/h = 16$, so the total number of attention parameters is the same as that of a single head with dimension $d = 64$.

2.4.3 Positional encoding

The attention operation eq. (2.7) is *permutation equivariant* in the input: shuffling the tokens of $\mathbf{Z}$ shuffles the output rows in the same way. For time series this is a problem: we explicitly want the network to know whether a feature occurred at the start or at the end of the window.

We follow Vaswani et al. [8] and add a fixed *sinusoidal positional encoding* to the token sequence before feeding it to the encoder:

$$\mathrm{PE}_{t,2k} = \sin\!\left(t \cdot \omega^{-2k/d}\right), \quad \mathrm{PE}_{t,2k+1} = \cos\!\left(t \cdot \omega^{-2k/d}\right), \quad \omega = 10\,000. \quad (2.9)$$

This vector is added to the token at position t before any attention layer is applied. Sines and cosines at geometrically decreasing frequencies let the network express both fine and coarse positional relations as easily-learned linear combinations.

2.4.4 The Transformer encoder block

The complete *Transformer encoder layer* composes multi-head self-attention with a two-layer feed-forward network and two LayerNorm [1] steps wrapped in residual connections (fig. 2.2):

$$\mathbf{Z}' = \mathbf{Z} + \mathrm{MultiHead}(\mathrm{LN}(\mathbf{Z})), \quad (2.10)$$

$$\mathbf{Z}'' = \mathbf{Z}' + \mathrm{FFN}(\mathrm{LN}(\mathbf{Z}')), \quad (2.11)$$

with $\mathrm{FFN}(x) = \mathbf{W}_2\,\mathrm{GELU}(\mathbf{W}_1 x + \mathbf{b}_1) + \mathbf{b}_2$. The pre-LayerNorm convention used here (LN *before* attention/FFN) is more stable to train than the original post-LN convention.

The CLS aggregation token. After the encoder we still have a length-T sequence, but we need a single class probability. Following BERT [2], we prepend to the sequence a special learnable vector — the *classification token* `[CLS]` — which has no signal content of its own but is

allowed to attend to every other token. After the encoder, the output row at the `[CLS]` position has aggregated information from the entire sequence and is fed alone to the linear classifier.

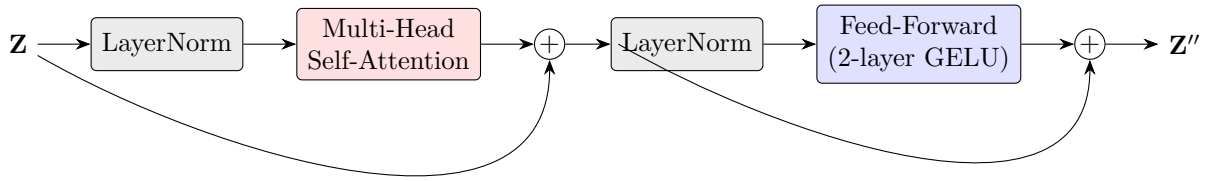

Figure 2.2. A pre-LN Transformer encoder block. Each sub-layer is wrapped in a residual connection; LayerNorm is applied before the sub-layer (pre-norm).

2.5 Training fundamentals

All three networks are trained with the same recipe.

2.5.1 Cross-entropy loss

For multi-class classification, the standard loss is the *categorical cross-entropy* introduced in eq. (2.2). Letting $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$ be the network's output logits and y the integer label, the contribution of a single example to the loss is

$$\mathcal{L}(\mathbf{x}, y) = -\log \frac{\exp(z_y)}{\sum_{c=1}^{K} \exp(z_c)}, \tag{2.12}$$

i.e. minus the logarithm of the probability the network assigns to the correct class after a softmax normalisation. The gradient of $\mathcal{L}$ with respect to z_c is $\mathrm{softmax}(\mathbf{z})_c - \mathbf{1}[c = y]$: the network is encouraged to push the correct logit up and pull the others down, in proportion to how confident the current (wrong) belief is.

2.5.2 Class weighting

When the training set is imbalanced, the loss is dominated by the majority class and minority classes are poorly learned. We compensate with a per-class weight (a per-example multiplier on the loss) inversely proportional to the class frequency,

$$w_c = \frac{N}{K \cdot n_c}, \tag{2.13}$$

where n_c is the count of class c in the training set and N is the total count. In our experiments this gives weights $\mathbf{w} = (0.46, 1.18, 1.27, 1.32, 1.24, 1.15, 1.31)$ for the seven classes: the `Normal` class is down-weighted by about a factor of two relative to the anomaly classes. We will see in chapter 7 that this choice, harmless on the synthetic test set, becomes problematic at deployment time because of the dramatic class prior shift in the real data.

2.5.3 Adam optimiser

The model is fit by stochastic gradient descent on mini-batches with the Adam optimiser [6], which maintains, in addition to the gradient, an exponential moving average of the first and second moments of the gradient per parameter and rescales the update accordingly:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \theta_{t+1} = \theta_t - \eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \tag{2.14}$$

where g_t is the gradient at step t and $\hat{m}_t, \hat{v}_t$ are the bias-corrected moments. The defaults $\beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$ are used. The initial learning rate is $\eta = 10^{-3}$.

2.5.4 Learning rate scheduling

A constant learning rate is rarely optimal: large steps early in training are useful to escape poor local minima, but late in training they oscillate around the minimum. We use `ReduceLROnPlateau`: whenever the validation loss has not improved for two consecutive epochs, the learning rate is multiplied by 0.5. This is the simplest form of adaptive schedule and is enough here to drive the loss down by roughly 20% in the last third of training.

2.5.5 Early stopping

To avoid over-fitting we monitor the validation loss and stop training when it has not improved by more than $\delta = 5 \cdot 10^{-3}$ for ten consecutive epochs. The model weights with the lowest validation loss are then re-loaded.

2.5.6 Temporal train/validation/test split

A subtle pitfall of sliding-window time-series classification is *data leakage*: if windows are extracted first and then shuffled across train/test, two windows that overlap in time can end up in different splits, so the test set effectively includes parts of the training signal. We avoid this by splitting the *raw signal* along time first (the first 80% becomes the train+validation pool, the remaining 20% becomes the test set), and only then applying the sliding-window extraction independently to each split. The 80% train+val pool is then further split uniformly at random into 80%/20%, which is safe because no overlap is possible anymore between training and validation windows on the same signal slice if the stride matches.

2.5.7 Window normalisation

Because the baseline of the signal drifts slowly (section 3.1), feeding the absolute dose to the network is inefficient: the network would waste capacity to learn the same features at every possible baseline. We *centre* each window independently — subtract its own mean — and divide by a single global standard deviation σ_{train} estimated on the centred training set:

$$\tilde{\mathbf{x}}^{(t)} = \frac{\mathbf{x}^{(t)} - \bar{\mathbf{x}}^{(t)}}{\sigma_{\text{train}}}. \tag{2.15}$$

The same σ_{train} is used at inference; saving it in a JSON file alongside the model weights guarantees deterministic and reproducible scaling between training and deployment. This window-level centring is a strong inductive bias — the network only ever sees deviations from the local baseline — but it has a hidden cost that becomes apparent in chapter 7.

Chapter 3

Data: real recordings and synthetic generator

This chapter describes the real-world data on which the project is grounded (section 3.1), the statistical model fitted to it (section 3.2), the parameterised anomaly templates that encode the radiological events we want to learn (section 3.3), and the generator that combines these ingredients into a large labelled training corpus (section 3.4).

3.1 Real 2015 gamma dose data

3.1.1 Source

The reference recordings are four months of gamma dose measurements collected in 2015 by an environmental surveillance beacon and provided by the project supervisor. The dates are

`24/02/2015 11:20` `30/04/2015 13:10` `22/06/2015 07:31` `20/10/2015 09:18`,

each segment lasting approximately 40 000 minutes (about 28 days) at one sample per minute. The four months are stored as separate columns in a single text file under `data-gen/data/` .

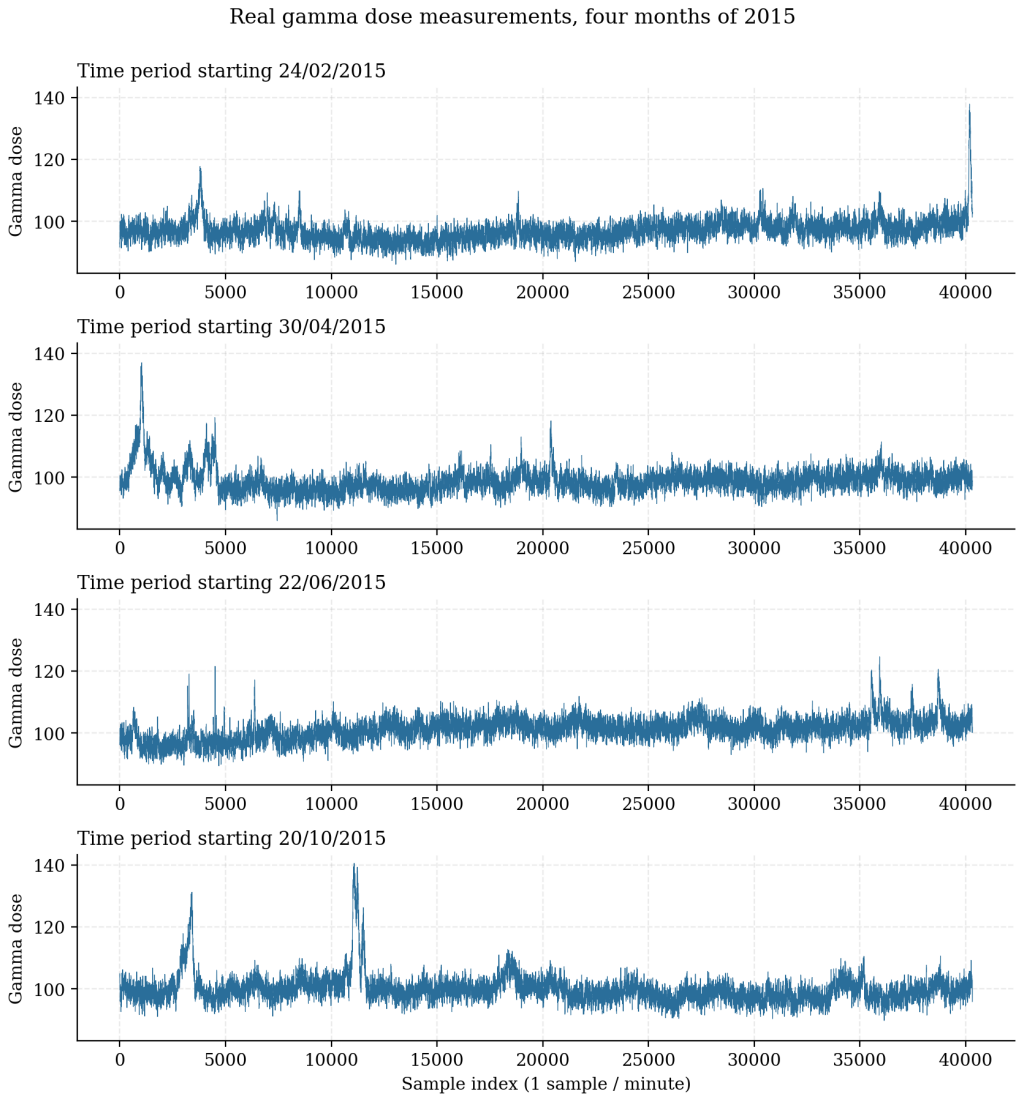

Figure 3.1. The four months of real gamma-dose recordings provided by the supervisor. Each panel shows a continuous segment of approximately one month at one sample per minute. The signal is centred around ~ 99 counts with a high-frequency dispersion of a few counts and slow drifts of a few units; a small number of sharper excursions are visible (the spike near sample 38 000 on February, the brief peak around sample 11 000 on October).

3.1.2 Statistical characterisation

Figure 3.1 suggests that the signal is the superposition of (i) a slowly-varying baseline and (ii) a high-frequency, zero-mean fluctuation. We confirm this by decomposing each month with a moving-average estimator of window length $w = 480$ samples (fig. 3.2):

$$b_t = \frac{1}{w} \sum_{i=-w/2}^{w/2} x_{t+i}, \qquad r_t = x_t - b_t. \tag{3.1}$$

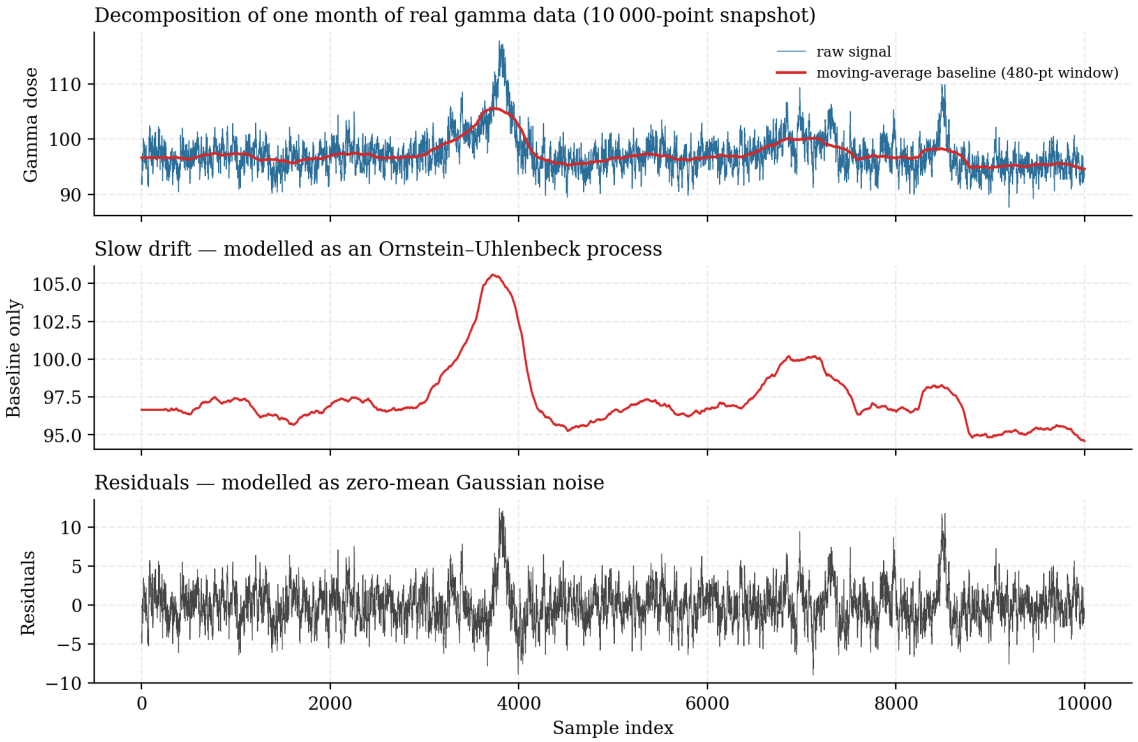

Figure 3.2. Decomposition of a 10 000-sample snapshot of the first month of real data into a slowly-varying baseline (red) and zero-mean residuals (bottom). The baseline is computed as a centred moving average with a $w = 480$-sample window. The residuals look stationary, zero-mean, and approximately Gaussian.

The empirical statistics, averaged across the four months, are reported in table 3.1. The baseline standard deviation is much smaller than the high-frequency residual standard deviation, but its *slow* variation is what motivates the random-walk baseline model used in the synthetic generator.

Month	Global mean μ	HF noise std σ	OU θ	OU σ_{OU}
24/02/2015	96.93	2.31	$4 \cdot 10^{-6}$	0.0277
30/04/2015	98.86	2.58	$4 \cdot 10^{-6}$	0.0390
22/06/2015	101.28	2.51	$4 \cdot 10^{-6}$	0.0337
20/10/2015	99.95	2.70	$4 \cdot 10^{-6}$	0.0485
Average across all	**99.26**	**2.53**	$\mathbf{4 \cdot 10^{-6}}$	**0.0373**

Table 3.1. Statistical characterisation of the four months of real gamma data, as extracted by `data-gen/evaluate_real_data.py`. The averaged values in the bottom row are those used in the synthetic generator.

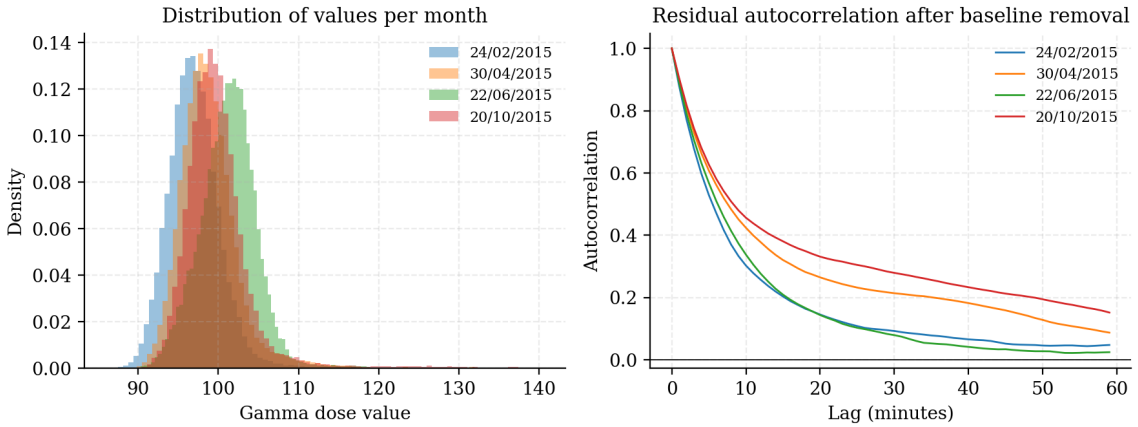

Figure 3.3. Left: empirical distribution of the raw gamma dose values per month. The distributions are unimodal and only slightly skewed, supporting a Gaussian approximation of the high-frequency noise. Right: autocorrelation of the residuals (after baseline removal) decays rapidly; remaining short-lag correlation indicates that the moving-average filter still leaks a small amount of drift into the residuals.

3.2 Noise model: Gaussian fluctuations on an OU baseline

Following the empirical findings of section 3.1.2, we model the real gamma dose as

$$x_t = b_t + \eta_t, \quad \eta_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2), \quad b_{t+1} = b_t + \theta(\mu - b_t) + \xi_t, \quad \xi_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma_{\text{OU}}^2). \tag{3.2}$$

The first equation says that the instantaneous reading is a baseline plus zero-mean Gaussian noise; the second is the Ornstein–Uhlenbeck (OU) discrete dynamics for the baseline. The parameter θ controls the mean-reversion strength: b_t is pulled towards the long-run mean μ at a rate θ, with random kicks of magnitude σ_{OU}. With $\theta = 4 \cdot 10^{-6}$, the baseline is very loosely tethered: the half-life of an excursion is roughly $\log 2/\theta \approx 1.7 \cdot 10^5$ samples, much longer than our analysis windows. This reproduces the slow drifts visible in fig. 3.1.

3.3 Anomaly templates

The six anomaly classes are described as *piecewise-interpolated parametric curves* stored as small CSV files in `data-gen/anomalies/`. Each row defines a *keypoint* with six fields plus an interpolation flag:

- `time` $\in [0, 1]$, `value` $\in [0, 1]$ — the position of the keypoint in normalised time and amplitude;
- `t_minus, t_plus, v_minus, v_plus` $\in \{0, 1\}$ — boolean flags that mark which quadrants the keypoint is allowed to move into when the template is jittered;
- `interp` $\in \{$`linear`, `exp_up`, `exp_down`, `bell`$\}$ — the interpolation mode for the segment that starts at this keypoint.

Figure 3.4 shows the six clean templates. Visually, they correspond to four common families of radiological signatures:

- **Smooth bumps** (`bell`, `bell_sq`): plumes that build up and decay smoothly, possibly with a small sustained plateau.
- **Asymmetric ramps** (`fast_ascend`, `fast_descend`): rapid onset followed by slow exponential decay, or its time-reverse.

- **Plateau** (`square`): a sustained contamination episode.
- **Multi-peak** (`M`): two close consecutive events — a useful adversarial shape for the network.

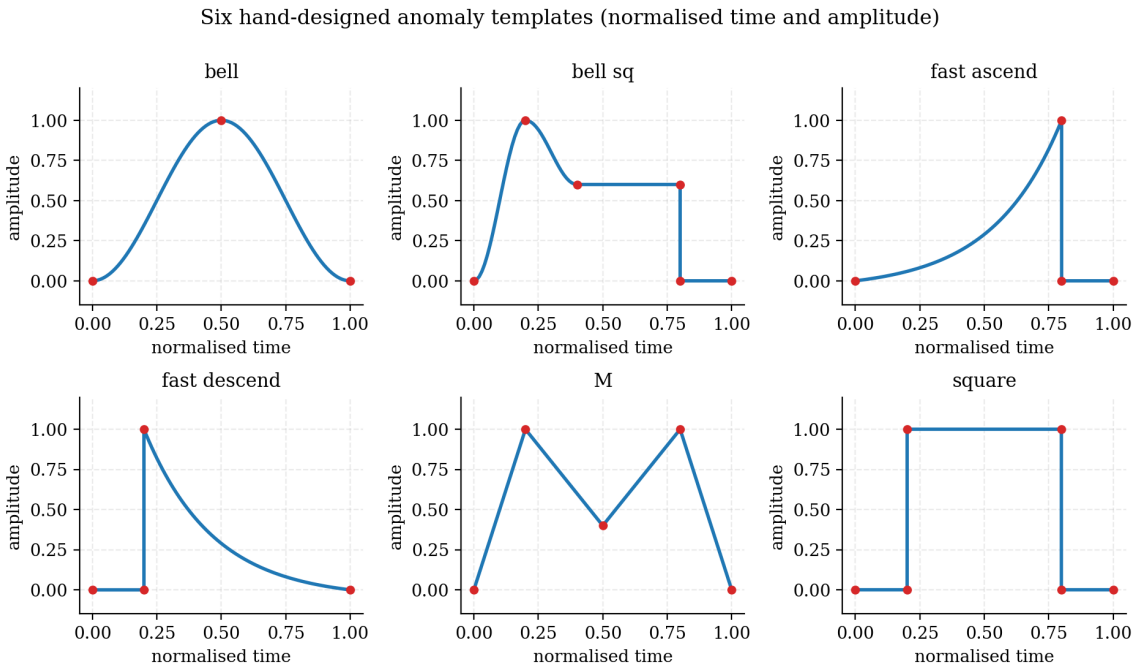

Figure 3.4. The six hand-designed anomaly templates, drawn in normalised time and amplitude. Each template is encoded as a small list of keypoints (red dots) connected by linear, exponential or half-cosine ("bell") segments.

3.3.1 Stochastic deformation of a template

A template would be useless on its own — training a network to recognise the exact same shape would not generalise. We therefore *deform* each template before injecting it into the signal. The algorithm, implemented in `anomaly_generator.py`, perturbs each keypoint independently by an isotropic Gaussian jitter of standard deviation ν (the *variance level*), then enforces the direction constraints encoded by the `t_minus`, ..., `v_plus` flags so that the deformed template still has the right qualitative shape (a peak cannot become a valley, time monotonicity is enforced as a fail-safe). The resulting deformed keypoints are then interpolated back into a dense curve and re-scaled to the target amplitude and period:

$$\text{amplitude} \in \sigma \cdot [\,2.5,\ 7.6\,], \qquad \text{period} \in [200,\ 800] \text{ samples}, \qquad \nu \in [0.02,\ 0.10]. \tag{3.3}$$

The amplitude is expressed in multiples of the high-frequency noise standard deviation σ, so that the per-sample signal-to-noise ratio of a generated anomaly lies in the interesting range $[2.5, 7.6]$.

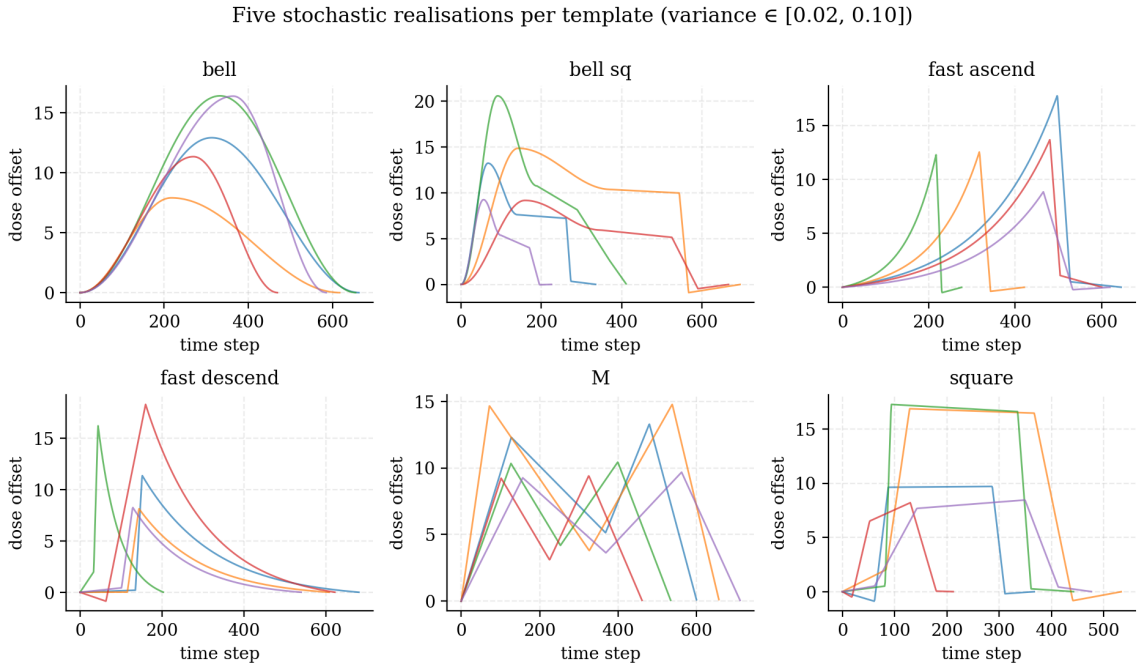

Figure 3.5. Five stochastic realisations of each template with random amplitude, period and deformation variance in the ranges used at training time. The qualitative shape is preserved, but no two realisations are identical.

3.4 Synthetic data generator

The end-to-end generator (`generate_custom_dataset.py`) combines the noise model of section 3.2 and the deformed templates of section 3.3.1 into a long labelled signal.

3.4.1 Algorithm

In pseudo-code, generating N points proceeds as follows:

1. Pick the total number of anomalies $M \in [5\,000, 8\,000]$ and pre-compute their evenly-spaced injection indices, with a small random jitter (± 200 samples) around each grid point.
2. For each injection point, sample a template uniformly at random, sample (amplitude, period, ν) from their ranges, and produce the deformed dense curve.
3. Generate the background in 50 000-point chunks: simulate the OU baseline path and overlay $\mathcal{N}(b_t, \sigma^2)$ noise.
4. Add each anomaly curve on top of the background at its injection index and label every output point with the anomaly's class (or 0 if the per-point contribution falls below a small threshold 10^{-2}).

The final dataset is written to disk in CSV chunks; it never lives entirely in RAM, which lets us produce datasets of arbitrary size within a constant memory budget.

3.4.2 Dataset used for this report

For this report we generate $N = 10^7$ samples and inject in the range $[5\,000, 8\,000]$ anomalies, leading to a global anomaly density of ~ 7 events per 10 000 samples. The signal looks as in fig. 3.6.

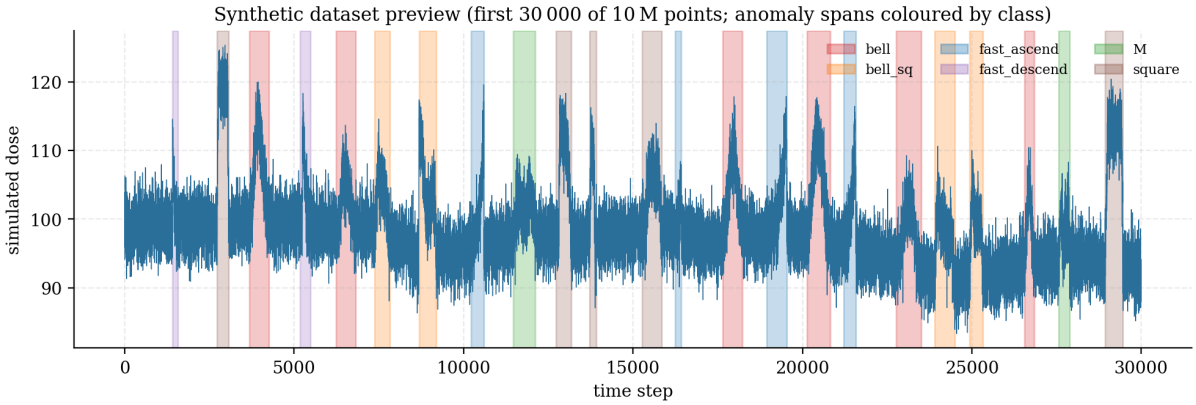

Figure 3.6. First 30 000 samples (of 10^7) of the generated synthetic dataset. Anomaly spans are coloured by class. The background standard deviation matches the real data of table 3.1; anomaly densities are deliberately higher than in the real recordings, so that the model is exposed to many examples of each class.

3.4.3 Why *this* generator?

Two design choices deserve a quick comment, because they will resurface in chapter 7.

Anomaly density is much higher than the real world. The synthetic dataset contains roughly one anomaly every 1 500 samples, whereas the real recordings contain a handful of plausible anomalies over 40 000 samples. This is a deliberate choice to give the classifier many examples of each class, but it skews the class prior seen by the network. We will see in section 7.2 that this is the largest single contributor to the model's failure on real data.

Templates assume a clear anomaly shape. The generator produces anomalies that are clean, isolated, and have a well-defined duration in $[200, 800]$ samples. Real-life events may be much shorter (a single contaminated sample) or much longer (a contamination episode that lasts hours), and the recordings also contain rapid, isolated single-point excursions that the templates do not represent at all. The trained classifier is therefore intrinsically blind to these out-of-template events.

Chapter 4

Architectures and training

This chapter is the engineering counterpart of chapter 2. Sections 4.1 to 4.3 describe the three neural networks we trained, with their exact layer composition and the intent behind each design choice. Section 4.4 summarises the three in a single comparison table. Section 4.5 then documents the shared training pipeline.

The three models share the same *interface*: input $\mathbf{x} \in \mathbb{R}^{1\times 512}$ (a single channel, a window of 512 samples), output $\hat{p} \in \mathbb{R}^7$ (probabilities over the seven classes). This makes them drop-in interchangeable for the training and inference scripts, which select a model by name and otherwise behave identically.

4.1 Architecture A — baseline 1D-CNN

The first model, defined in `architectures/saved_models/1d_cnn.py`, is a deliberately compact convolutional network. It serves both as a baseline to beat and as a sanity check that the data and the training pipeline are correctly wired.

Structure. The network is a linear stack of three convolution–BatchNorm–ReLU–MaxPool blocks, followed by an adaptive average pool and a small classifier head:

Block	**Operation**	**Output shape**
Input	Raw window	$(1, 512)$
Block 1	Conv1D($1\to16, k{=}7, p{=}3$) $\to$ BN $\to$ ReLU $\to$ MaxPool(2)	$(16, 256)$
Block 2	Conv1D($16\to32, k{=}7, p{=}3$) $\to$ BN $\to$ ReLU $\to$ MaxPool(2)	$(32, 128)$
Block 3	Conv1D($32\to64, k{=}5, p{=}2$) $\to$ BN $\to$ ReLU $\to$ MaxPool(2)	$(64, 64)$
Pool	AdaptiveAvgPool1d(4)	$(64, 4)$
Classifier	Flatten $\to$ Dropout(0.4) $\to$ Linear($256\to32$) $\to$ ReLU $\to$ Dropout(0.2) $\to$ Linear($32\to7$)	(7)

Design notes.

- Kernel sizes $k = 7, 7, 5$ are large enough to capture multi-sample shape primitives (ramps, peaks) at each scale, while staying well within the receptive-field budget.
- Channel widths $16 \to 32 \to 64$ double at each block, compensating the temporal compression by a factor of 2 at every `MaxPool`.
- `AdaptiveAvgPool1d(4)` keeps four temporal bins after the convolutional trunk: this preserves a coarse notion of "where in the window the activation peaks", a useful piece of information

for shapes such as `fast_ascend` vs `fast_descend` that mainly differ by the location of their peak.

- Two dropout layers ($p = 0.4$ then $p = 0.2$) in the fully-connected head act as a regulariser and prevent the small head from overfitting the convolutional features.

Total trainable parameters: 22 727.

4.2 Architecture B — 1D Residual Network

The second model, defined in `architectures/saved_models/resNet.py`, replaces each convolutional block of model A with a small *residual block*, as introduced in section 2.3. It also replaces every BatchNorm with GroupNorm (section 2.2.5).

Residual block. A single block transforms an input of channel count c_{in} into an output of channel count c_{out} as

$$\begin{aligned} \mathcal{F}(\mathbf{u}) &= \mathrm{GN} \circ \mathrm{Conv1D}_{k_3} \circ \mathrm{ReLU} \circ \mathrm{GN} \circ \mathrm{Conv1D}_{k_2} \circ \mathrm{ReLU} \circ \mathrm{GN} \circ \mathrm{Conv1D}_{k_1}(\mathbf{u}), \\ \mathrm{Block}(\mathbf{u}) &= \mathrm{ReLU}\big(\mathcal{F}(\mathbf{u}) + \mathrm{Shortcut}(\mathbf{u})\big), \end{aligned}$$

where the shortcut is either the identity (when $c_{\text{in}} = c_{\text{out}}$) or a 1×1 convolution followed by GroupNorm. The kernel sizes (k_1, k_2, k_3) are $(15, 9, 5)$ in the first block and $(7, 5, 3)$ in the subsequent two; the first block uses a larger initial kernel to capture broad context immediately.

Block	Operation	Output shape
Input	Raw window	$(1, 512)$
Block 1	$\mathrm{Res}\big(c_{\text{in}}{=}1 \to c_{\text{out}}{=}16,\ k{=}(15,9,5)\big)$	$(16, 512)$
Pool 1	MaxPool(2)	$(16, 256)$
Block 2	$\mathrm{Res}\big(16 \to 32,\ k{=}(7,5,3)\big)$	$(32, 256)$
Pool 2	MaxPool(2)	$(32, 128)$
Block 3	$\mathrm{Res}\big(32 \to 64,\ k{=}(7,5,3)\big)$	$(64, 128)$
Pool	AdaptiveAvgPool1d(4)	$(64, 4)$
Classifier	Same head as model A	(7)

Full structure.

Design notes.

- GroupNorm with $\min(8, c_{\text{out}})$ groups everywhere means train-time and inference-time behaviour are identical, which matters when deploying on real data with a different value distribution.
- The shortcut connections preserve the original signal through the convolutional trunk, allowing later blocks to focus on *adding* discriminative information rather than rediscovering the input.
- The classifier head is intentionally identical to that of model A, so that the residual trunk is the only difference between the two networks — a clean ablation.

Total trainable parameters: 74 631.

4.3 Architecture C — CNN+Attention

The third model, defined in `architectures/saved_models/cnn_attention.py`, hybridises a convolutional stem with a Transformer encoder (section 2.4.4). The motivation is to combine the *locality* of convolutions, which is well-suited to extracting short shape primitives, with the *global mixing* of attention, which lets the network reason about how those primitives compose over the whole window.

Convolutional stem. Three Conv–GroupNorm–ReLU blocks reduce the temporal length by a factor of 8 via stride-2 convolutions (so no explicit `MaxPool` is needed):

Layer	Operation	Output shape
Input	Raw window	$(1, 512)$
Stem 1	Conv1D($1\to16, k{=}7, s{=}2, p{=}3$) $\to$ GN(8) $\to$ ReLU	$(16, 256)$
Stem 2	Conv1D($16\to32, k{=}5, s{=}2, p{=}2$) $\to$ GN(8) $\to$ ReLU	$(32, 128)$
Stem 3	Conv1D($32\to64, k{=}3, s{=}2, p{=}1$) $\to$ GN(8) $\to$ ReLU	$(64, 64)$

Tokenisation. The stem output is reshaped to a sequence of $T = 64$ tokens of dimension $d = 64$. We prepend a learnable `[CLS]` token, giving $T' = 65$ tokens, and add a sinusoidal positional encoding (eq. (2.9)).

Transformer encoder. Two pre-LayerNorm encoder blocks (fig. 2.2) are stacked. Each block has $h = 4$ attention heads of dimension $d_h = 16$ and a feed-forward network with hidden size $d_{\text{ff}} = 4 \cdot d = 256$, GELU activation and dropout $p = 0.1$.

Classification. After the encoder, the `[CLS]` row of the output is LayerNorm-normalised and passed through a single linear layer $\mathbb{R}^d \to \mathbb{R}^7$ to produce the class logits.

Design notes.

- The stride-2 convolutional stem reduces the sequence length eightfold before attention is applied, which keeps the $O(T^2)$ cost of attention manageable at $65^2 \approx 4\,000$ pairs.
- GroupNorm (not BatchNorm) in the stem makes the entire feature extraction amplitude-invariant on a per-window basis — consistent with the window-mean centring of eq. (2.15).
- The `[CLS]` token is initialised with a small random vector (truncated normal, $\sigma = 0.02$) and is the only *learnable* positional information; the sinusoidal encoding is shared and fixed.

Total trainable parameters: 109 655.

4.4 Comparison of the three architectures

Table 4.1 summarises the three architectures side by side. The trend is clear: each step adds expressive power at the cost of more parameters but *not* much more training time, because the deeper architectures still operate on heavily down-sampled features.

	1D-CNN	ResNet 1D	CNN+Attention
Convolutional trunk	3 plain blocks	3 residual blocks	3-layer stride-2 stem
Normalisation	BatchNorm	GroupNorm(8)	GroupNorm(8)
Skip connections	—	yes	yes (inside Transformer)
Long-range mixing	—	—	Multi-head self-attention
Output head	MLP($256 \to 32 \to 7$)	MLP($256 \to 32 \to 7$)	Linear($64 \to 7$) on CLS
Trainable parameters	22 727	74 631	109 655
Training time (35 epochs, RTX 5060)	$4'18''$	$10'02''$	$10'14''$
Inference throughput	10 664 win/s	14 232 win/s	15 184 win/s

Table 4.1. Side-by-side summary of the three architectures. Throughput is measured on the same RTX 5060 Laptop GPU at a stride of 10 samples between successive windows.

4.5 Training pipeline

The training pipeline is implemented once, in `architectures/train.py`, and parameterised by the model name.

4.5.1 Data preparation

The raw signal is split temporally (80/20) into a train+val pool and a test pool (section 2.5.6), then converted into sliding windows of size 512 with stride 32:

- Train+val pool: 249 985 windows (random 80/20 split: 199 988 train, 49 997 validation).
- Test pool: 62 485 windows.

Each window is centred individually and scaled by the global training standard deviation $\sigma_{\text{train}} = 4.21$ (eq. (2.15)). σ_{train} is saved in a JSON file next to the model weights so that inference uses exactly the same scaling.

Window class distribution. The labelling rule of section 2.1 yields roughly 31% of windows of class `Normal` and 11%–12% of each of the six anomaly classes (table 4.2).

Class	Count	Share	Class weight w_c
`Normal Background`	77 674	31.1%	0.46
`bell`	30 218	12.1%	1.18
`bell_sq`	28 106	11.2%	1.27
`fast_ascend`	27 004	10.8%	1.32
`fast_descend`	28 769	11.5%	1.24
`M`	30 963	12.4%	1.15
`square`	27 251	10.9%	1.31
Total	249 985	100%	—

Table 4.2. Window-level class distribution in the training set and the resulting class weights used in the cross-entropy loss.

4.5.2 Optimisation

All three models are trained with the recipe of section 2.5: Adam ($\eta = 10^{-3}$, default β_1, β_2), batch size 128, weighted cross-entropy eq. (2.2), a `ReduceLROnPlateau` scheduler with patience 2 and factor 0.5, early stopping with patience 10 and minimum delta $5 \cdot 10^{-3}$, maximum 35 epochs. No data augmentation is applied — the stochastic anomaly generation already provides enough variability.

4.5.3 Selection criterion

The training script saves the weights with the lowest validation loss. Validation F1 score (macro-averaged across the seven classes) is tracked at every epoch for diagnostic purposes but is not used for selection.

Chapter 5

Results on synthetic data

This chapter reports the quantitative performance of the three architectures of chapter 4 on the held-out synthetic test segment. We look at the training dynamics (section 5.1), the sample-level (window-level) accuracy (section 5.2), and the more operationally meaningful event-level metrics (sections 5.3 and 5.4). The chapter closes with a cost/performance comparison (section 5.5).

5.1 Training dynamics

Figure 5.1 overlays the train and validation losses and the validation macro F1 of all three models. Several observations:

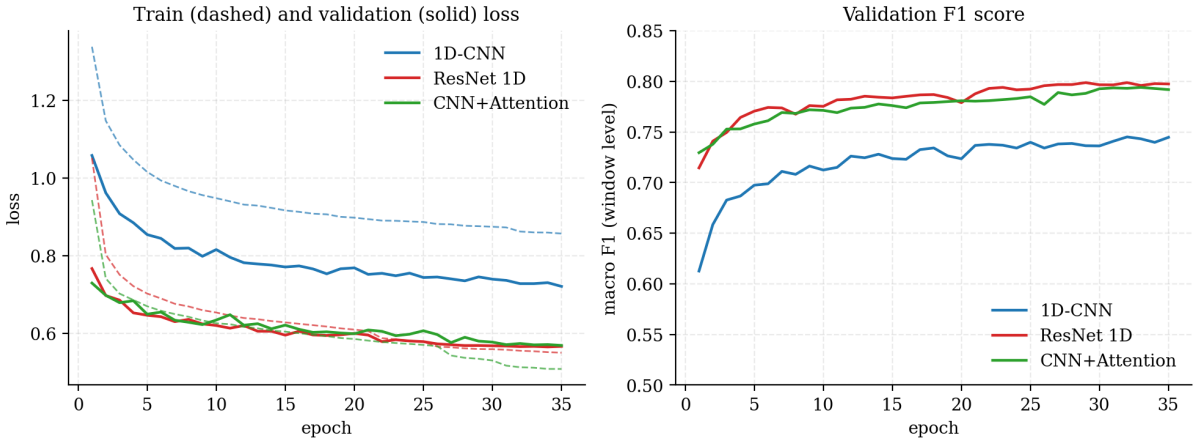

Figure 5.1. Training dynamics. Left: train loss (dashed) and validation loss (solid). Right: validation macro F1 (window level). All three models converge within 35 epochs and benefit from one or two learning-rate reductions; the residual and attention models reach visibly lower loss plateaus than the baseline.

- The 1D-CNN converges to a higher loss plateau (≈ 0.72 validation loss) than both the ResNet and the attention model (≈ 0.57). The gap between train and validation curves remains small for all three networks, indicating that none of them is severely over-fitting: the residual block and the attention block do extract genuinely more useful features rather than memorising the training set.
- The validation F1 starts much higher for the residual and attention models (above 0.71 at the first epoch) than for the baseline (0.61). The residual variant is the fastest to converge to its plateau — the skip connections seem to make optimisation not only easier but also faster.

- The attention model's curve is the most non-monotonic — consistent with the higher variance of Transformer training — but the `ReduceLROnPlateau` schedule cuts the learning rate twice and stabilises it.

5.2 Window-level test performance

After training, each model's best checkpoint is loaded and evaluated on the held-out test windows. Table 5.1 reports the test accuracy and per-class accuracy.

Class	**1D-CNN**	**ResNet 1D**	**CNN+Attention**
`Normal Background`	0.941	0.942	0.911
`bell`	0.697	0.779	0.765
`bell_sq`	0.619	0.713	0.712
`fast_ascend`	0.704	0.737	0.744
`fast_descend`	0.721	0.741	0.738
`M`	0.687	0.765	0.766
`square`	0.703	0.740	0.732
Overall accuracy	0.766	0.807	0.795

Table 5.1. Window-level accuracy on the held-out synthetic test set (62 485 windows). The residual network is the strongest on the anomaly classes; the attention model is slightly weaker on the `Normal` class.

These numbers look modest — roughly 20% of windows are misclassified — but they are deeply misleading at the operational level. The reason is that a window is labelled according to the *majority anomaly class* inside it, but a single anomaly typically spans many overlapping windows that include large fractions of normal background. Windows near the boundary of an event easily flip between the anomaly class and `Normal` depending on whether they pass the 10% threshold. What we really care about is whether the model identifies the event *as a whole*.

5.3 Event-level evaluation methodology

We therefore evaluate the system on a continuous segment using a *detection-driven* protocol implemented in `architectures/inference_engine.py`.

5.3.1 Continuous probability trace

A test segment of 100 000 samples (immediately following the training portion) is fed to the network through a sliding window of size 512 and stride 10. Each window output produces a probability vector of length $K = 7$ which is averaged into the per-sample running total (using $1/n_{\text{visits}}$ as weight). At the end we have a sample-level probability matrix $\hat{P} \in \mathbb{R}^{100\,000 \times 7}$.

5.3.2 Predicted-event extraction

A sample is declared "anomalous" if $\hat{P}_{t,0} < 0.3$ (i.e. the network is at least 70% confident that this sample is *not* normal). Contiguous regions of anomalous samples form predicted events; small gaps of up to 50 samples are bridged to avoid splitting a single event into multiple fragments.

5.3.3 Class assignment for each event

The class of a predicted event is decided by *integrating* the probability vectors over the event's duration and picking the class with the highest cumulative score:

$$\hat{y}_{\text{event}} = \underset{c \in \{1,\dots,K-1\}}{\arg\max} \sum_{t \in \text{event}} \hat{P}_{t,c}. \tag{5.1}$$

This averaging makes the classification much more robust to per-sample noise than a majority vote on per-sample arg max predictions.

5.3.4 Matching predicted to true events

True events are extracted from the ground-truth labels by the same contiguous-region procedure. A predicted event is matched to a true event whenever the two intervals overlap; the match with the largest overlap is kept. The result is split into:

- *True positives (TP)*: a true event matched to a predicted event with the correct class.
- *Misclassifications*: a true event matched to a predicted event with the wrong class (counted as a false *negative* for the true class).
- *Missed events (FN)*: a true event with no overlapping prediction.
- *Hallucinations (FP)*: a predicted event that does not overlap any true event.

Per-class precision, recall and F1 are then computed in the usual way, and the report quotes the macro average across the six anomaly classes.

5.4 Event-level results

On the 100 000-sample test segment we have 59 ground-truth events. The event-level confusion matrices for the three models are shown in fig. 5.2.

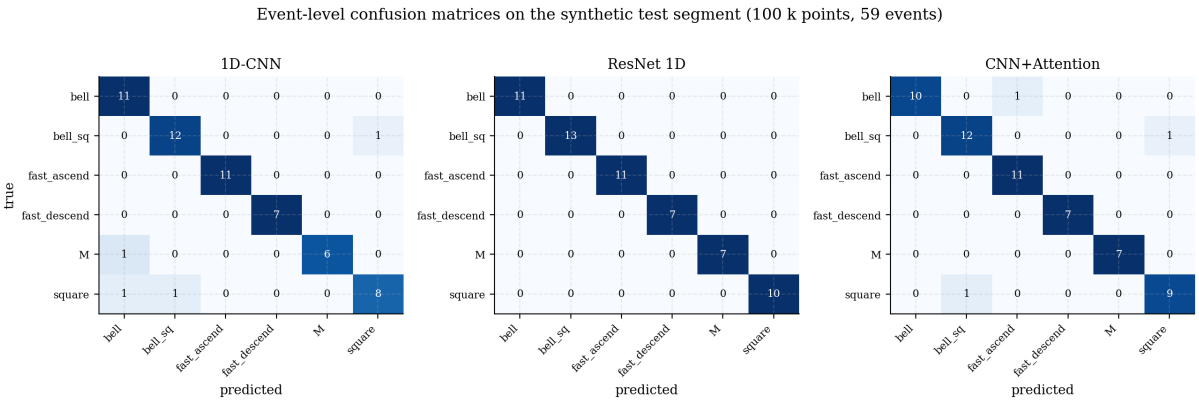

Figure 5.2. Event-level confusion matrices on the synthetic test segment. Cell colour is the row-normalised recall; the integer inside each cell is the raw event count. The residual network achieves a perfect classification; the attention model misses three events (−1 on `bell`, −1 on `bell_sq`, −1 on `square`); the baseline 1D-CNN misses four events, with the typical confusions `M` → `bell` and `square` → `bell_sq`.

The per-class precision, recall and F1 are reported in table 5.2.

	1D-CNN			ResNet 1D			CNN+Attention		
Class	P	R	F1	P	R	F1	P	R	F1
`bell`	0.85	1.00	0.92	1.00	1.00	1.00	1.00	0.91	0.95
`bell_sq`	0.92	0.92	0.92	1.00	1.00	1.00	0.92	0.92	0.92
`fast_ascend`	1.00	1.00	1.00	1.00	1.00	1.00	0.92	1.00	0.96
`fast_descend`	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
`M`	1.00	0.86	0.92	1.00	1.00	1.00	1.00	1.00	1.00
`square`	0.89	0.80	0.84	1.00	1.00	1.00	0.90	0.90	0.90
Macro avg.	**0.94**	**0.93**	**0.93**	**1.00**	**1.00**	**1.00**	**0.96**	**0.96**	**0.96**

Table 5.2. Per-class precision (P), recall (R) and F1 at the event level on the synthetic test segment. The residual model is flawless on this segment; the attention model achieves 0.96, and the baseline 0.93.

Reading the numbers. A 93% event-level F1 on the baseline is already very useful: it means that out of the 59 events in a 100 000-sample segment, the baseline correctly identifies 55, misclassifies 4, and hallucinates none. The residual model identifies all 59 events correctly without hallucinating. The attention model is competitive but produces two confusions and one missed-then-misclassified event.

Where errors live. The two recurring confusions are between templates that share visual primitives:

- `M` can look like `bell` if its second peak is eroded by deformation;
- `square` can look like `bell_sq` if the corners are softened.

The residual network, with its larger receptive field and richer feature re-use, never makes these confusions on the test segment; the baseline and the attention model trip on a few hard examples.

No hallucinations. A striking feature of all three models on this clean test set is that the false-positive rate is essentially zero: predicted events that do not overlap a true event are extremely rare. We will see in chapter 7 that this property collapses dramatically on real, almost-pure-background data.

5.5 Cost and performance trade-off

Figure 5.3 summarises the cost–performance trade-off across the three architectures. Note the very small spread in training time between ResNet and CNN+Attention (about 10 minutes for 35 epochs) despite the latter having 47% more parameters, reflecting the efficiency of GPU-resident attention on short sequences ($T = 65$ tokens).

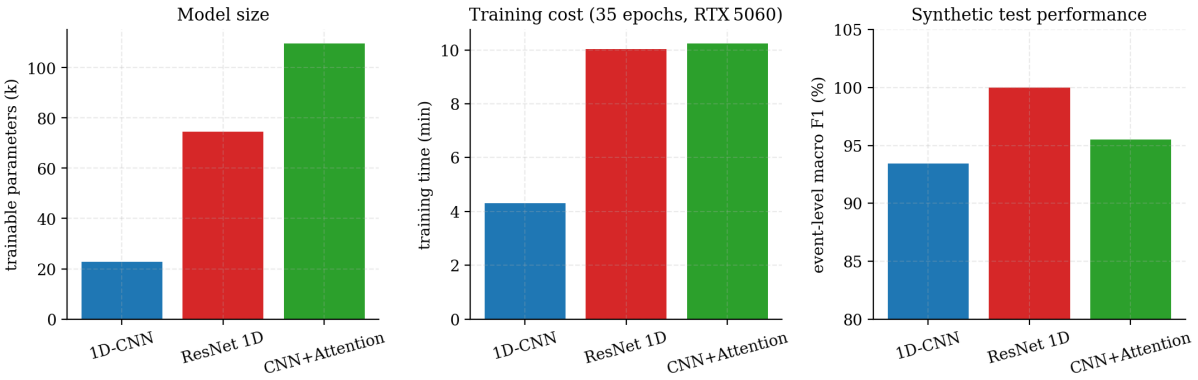

Figure 5.3. Cost–performance trade-off across the three architectures. Left: parameter count. Centre: total training time on a single RTX 5060 Laptop GPU. Right: event-level macro F1 on the synthetic test segment. The ResNet is, on this benchmark, the sweet spot: half the parameters of the attention model, a perfect F1, and similar wall-clock time.

Chapter 6

Robustness study

The numbers reported in chapter 5 all assume that the deployment distribution matches the training distribution. In practice, a deployed beacon will see varying noise conditions and anomaly shapes that the templates do not capture exactly. This chapter quantifies how gracefully each architecture degrades when the deployment data is forced *out of distribution* along two controlled axes.

6.1 Protocol

The robustness script (`architectures/robustness_test.py`) generates a series of small test datasets, each one with 50 injected events, varying a single distortion parameter while holding the other at its baseline value:

1. **Background-noise sweep.** The high-frequency noise standard deviation σ is multiplied by $m_{\text{noise}} \in \{1.00, 1.22, 1.44, 1.67, 1.89, 2.11, 2.33, 2.56, 2.78, 3.00\}$. To make the sweep a fair test of *absolute* robustness rather than a constant SNR test, the target amplitude range of injected anomalies is divided by m_{noise}, keeping the absolute strength of the anomaly constant.
2. **Anomaly-shape-deformation sweep.** The deformation-variance parameter ν of the template generator (see section 3.3.1) is multiplied by $m_{\text{deform}} \in \{1.00, 1.78, 2.56, \ldots, 8.00\}$ (ten evenly-spaced values between 1 and 8). Background noise is held at its training value.

Each generated dataset is then passed through the trained model and evaluated with the event-level protocol of section 5.3. The metric reported is the macro F1 across the six anomaly classes.

6.2 Results

Figure 6.1 overlays the three architectures' macro F1 degradation curves on a common scale; the dotted horizontal line marks the F1 = 50% threshold below which the model is no longer reliably useful.

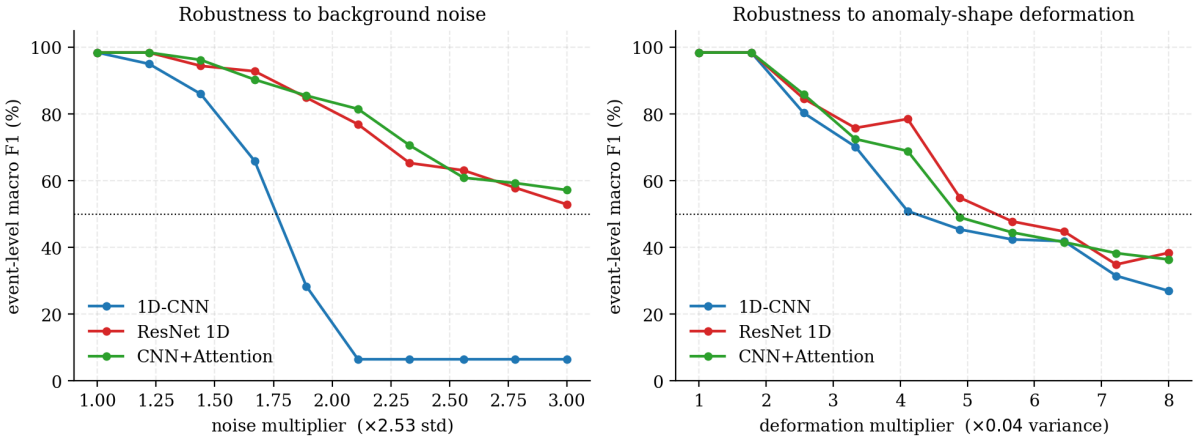

Figure 6.1. Robustness curves. **Left**: event-level macro F1 as a function of the background-noise multiplier. **Right**: macro F1 as a function of the anomaly-shape deformation multiplier. The dotted line marks 50% F1.

Background-noise sweep.

- The **1D-CNN** collapses dramatically: its F1 drops below 50% at $m_{\text{noise}} \approx 1.9$ and reaches the floor of $\sim 7\%$ already at $m_{\text{noise}} = 2.1$, where the model essentially predicts a single class for the entire signal.
- The **ResNet** and the **CNN+Attention** *never* dip below 50% on this sweep — even at $m_{\text{noise}} = 3.0$, where the noise standard deviation is three times what the model was trained on, they retain 53% and 57% F1 respectively. The two are essentially indistinguishable on this axis.

Anomaly-shape-deformation sweep.

- All three models degrade smoothly and only fall below 50% F1 at deformation multipliers of $\sim 5{-}6$, i.e. when the keypoint jitter exceeds 20%−30% of the template's normalised height — a level at which a human would also struggle to assign a class.
- The **ResNet** is the most resilient to extreme deformation, maintaining $\sim 45\%$ F1 up to $m_{\text{deform}} = 6.4$ where the other two have dropped below 42%. The **CNN+Attention** is essentially on par.
- The **1D-CNN** is the first to break, falling below 50% at $m_{\text{deform}} \approx 4.9$.

6.3 Discussion

Two structural conclusions can be drawn from the robustness study.

1. Residual connections are the single biggest win. The gap between the baseline 1D-CNN and the ResNet is enormous on the noise axis (~ 50 F1 points at $m_{\text{noise}} = 2.0$) and significant on the deformation axis. The two architectures have an otherwise identical recipe (same trunk depth, same output head, same input window). The only differences are (i) residual connections and (ii) the substitution of BatchNorm by GroupNorm. We can disentangle these in principle by training a third ablation, but both effects point in the same direction: a network whose features survive a small distribution shift is structurally important for an operational beacon.

2. Self-attention does not buy extra robustness here. On both axes the CNN+Attention model is essentially tied with the ResNet. The Transformer encoder layers add capacity for long-range reasoning, but the windows in this problem are short (512 samples, $\sim$ 8.5 hours), so attention does not have much extra context to exploit. On a much longer window — say a 24-hour window with several events of interest — we would expect the attention model to pull ahead. With the current $W = 512$ setup, the attention block essentially behaves like an additional global-pooling step, and the extra cost is not amortised.

3. The flat-line failure of the 1D-CNN at high noise. The most striking artefact in fig. 6.1 (left) is the plateau at F1 $\approx 6.5\%$ for the 1D-CNN above $m_{\text{noise}} = 2.1$. A macro F1 of 6.5% on six classes corresponds approximately to the model predicting a *single* non-trivial class for the whole signal: it correctly identifies all the events of that class (perfect recall) but misses all the others entirely, leaving precision and recall on the other five classes at zero. This is a hallmark of a classifier that has lost confidence in its features and falls back to the largest receptive bias. The same failure mode is the one we will diagnose on real data in chapter 7.

Operational implications. A beacon that operates over years sees seasonal variations in background noise that easily exceed a $\times 1.5$ multiplier. The robustness data argue that the residual or attention architecture is the only viable choice if the system has to keep functioning across seasons without retraining.

Chapter 7

Inference on real 2015 data

The synthetic results of chapter 5 and the robustness curves of chapter 6 suggest that, on its own training distribution, the system is in good shape. This chapter confronts the trained models with the *real* 2015 gamma-dose recordings from which the synthetic generator was originally calibrated. The outcome is sobering: all three models behave poorly on the real signal, and each fails in its own characteristic way. We document the symptoms (section 7.1), then walk through the most plausible causes (sections 7.2 to 7.6) and close with a concrete list of remediation steps for the next iteration of the project (section 7.9).

7.1 Symptoms

The script `architectures/inference_real_data.py` runs the same sliding-window inference pipeline used for the synthetic test segment on each of the four months of real data (section 3.1), then overlays the smoothed per-class confidence on the raw signal. Figure 7.1 shows the first 10 000 samples of February 2015 for each of the three models.

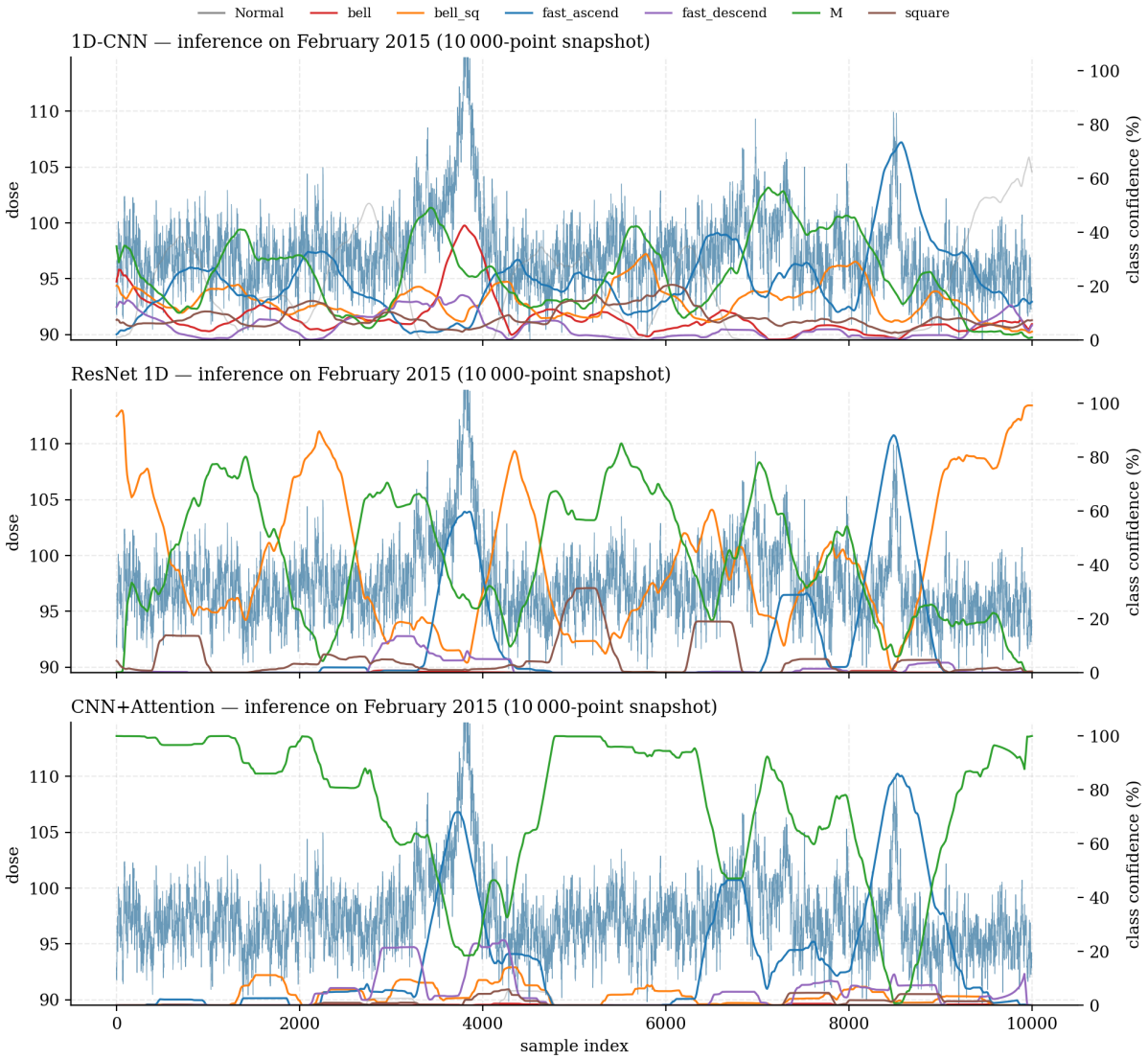

Figure 7.1. Real-data inference on the first month, 10 000-sample snapshot, for each architecture. Blue: raw gamma dose. Coloured lines: per-class confidence (right axis, 0–100%), smoothed with a 30-sample rolling mean. The grey curve is the `Normal Background` confidence; the coloured curves are the anomaly classes. None of the three models behaves the way it does on synthetic data.

1D-CNN: high-frequency oscillating predictions. The 1D-CNN never commits to any one class for long: each anomaly class takes its turn going up and down between 0% and $\sim 40\%$. The `Normal Background` probability sits around 50%–70% for most of the window but never gets close to the operational threshold of $\sim 70\%$ that would make the system silent. There is a clear spike in `M` confidence around sample 4 000 that coincides with the visible bump in the raw signal — this is encouraging — but the same level of confidence is reached elsewhere, in places where nothing is happening.

ResNet: persistent class biases, sustained false-positive plateaus. The ResNet does not oscillate; instead it locks onto one anomaly class at a time for long stretches. The orange `bell_sq` confidence plateaus near 100% for thousands of samples; the green `M` confidence does the same on the second half of the window. The model is essentially producing a continuous stream of false positives, silently classifying every quiet region of the signal as one anomaly or another.

CNN+Attention: degenerate collapse to a single class. The hybrid model exhibits the most dramatic failure: the green `M` confidence is essentially pinned to 100% for the whole 10 000-sample snapshot. The signal could be flat at the global mean, and the model would still declare `M` with full confidence. This is the same kind of collapse we observed at the high-noise extreme of the robustness sweep (section 6.3): when the input is outside what the model has learned to discriminate, it falls back to the class whose "signature" best matches anything.

The three failures share a single root cause — distribution shift — but they are amplified differently by each architecture. We now look at the five ingredients that make this shift so large.

7.2 Cause 1: class-prior shift between training and deployment

This is by far the most important effect.

What the model saw at training time. The synthetic dataset (section 3.4.2) was designed to expose the network to many examples of each anomaly type. The result is a class distribution in which only $\sim 31\%$ of windows are `Normal Background` and the other six classes share the remaining 69% roughly evenly (table 4.2).

What the model sees at deployment. The real recordings contain at most a handful of plausible anomalies over 40 000 samples. Even on the most "eventful" month, $> 99\%$ of windows are essentially pure background. The deployment prior is therefore drastically skewed towards `Normal`, and exactly in the opposite direction of the training prior (fig. 7.2, left).

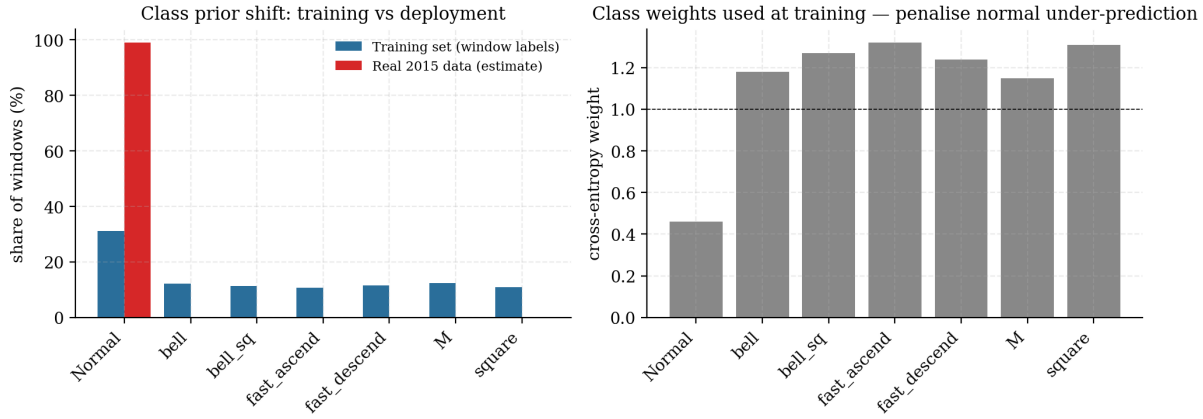

Figure 7.2. Class-prior shift between training and deployment. **Left**: window-level class distribution observed during training (blue) versus a rough estimate of the same distribution on the real 2015 data (red). **Right**: cross-entropy class weights w_c used during training. The weight on `Normal Background` is roughly half that of any anomaly class, which actively biases the network towards predicting an anomaly at deployment.

The compounding effect of class weights. On top of the prior shift, we used class weights in the cross-entropy loss (section 2.5.2) that down-weight the `Normal` class by a factor of $w_0/\bar{w}_{\text{anom}} \approx 0.46/1.25 \approx 0.37$. From a Bayesian point of view, this is equivalent to *further multiplying* the network's posterior on the anomaly classes by $\sim 1/0.37 \approx 2.7$. In a balanced training set this is harmless — it merely corrects an under-representation of the minority anomaly classes inside the training distribution. But at deployment, where the true prior of `Normal` is already $\gg 95\%$, this multiplier acts like a built-in trigger-happy bias.

Quantitative reading. Let $p_{\text{tr}}(y)$ be the training prior and $p_{\text{dp}}(y)$ be the deployment prior. The Bayes-optimal classifier on the deployment distribution should rescale the network's output as

$$\hat{p}_{\text{dp}}(y \mid \mathbf{x}) \propto \hat{p}_{\text{net}}(y \mid \mathbf{x}) \cdot \frac{p_{\text{dp}}(y)}{p_{\text{tr}}(y)}, \tag{7.1}$$

where the ratio for `Normal` is $\sim 0.99/0.31 \approx 3.2$ and for any anomaly class is $\sim 0.002/0.115 \approx 0.017$. We should be multiplying the `Normal` posterior by ~ 3 and shrinking each anomaly posterior by $\sim 60\times$. We instead did the opposite (we down-weighted `Normal` at training), so the deployed model is mis-calibrated by a factor of roughly 200.

7.3 Cause 2: anomaly-duration mismatch

The synthetic templates have durations in $[200, 800]$ samples, a range chosen to make the windowed classification problem well posed (the anomaly must occupy at least 10% of a 512-sample window to be labelled, see section 2.1). The real signal contains at least two qualitatively different anomaly families that the templates do not represent:

- **Single-sample excursions.** The Fevereiro recording contains a spike from ~ 100 to ~ 138 at the end of the month that lasts a single sample. Even if this is a sensor glitch rather than a true radiological event, the trained models have never seen a 1-sample anomaly and there is no class they can reasonably assign to it.
- **Multi-hour contamination episodes.** Conversely, a real plume contamination could last 1 000 samples or more. Once an event lasts longer than the analysis window (512 samples), the windowing approach loses its ability to "see" the event's edges and the per-window centring (section 7.5) removes the event amplitude.

The templates we trained on therefore cover a relatively narrow band of the true anomaly-duration distribution. Anything outside that band either passes for normal or, more dangerously, is force-matched to the closest template.

7.4 Cause 3: baseline-drift mismatch

The Ornstein–Uhlenbeck baseline used in the generator is fitted on the four months of real data, but the parameters $(\theta, \sigma_{\text{OU}})$ reflect only the *average* behaviour, and the OU model produces a very smooth, small-amplitude random walk. The real recordings, by contrast, show visible *trends* (fig. 3.1): the Fevereiro signal rises by ~ 5 units over 20 000 samples, the Outubro signal rises by ~ 7 units over a similar span. These slow systematic drifts are not Gaussian random walks; they are likely seasonal and meteorological. The trained model has not seen this kind of amplitude-modulated baseline, so a 512-sample window taken during a trend can have an internal slope that the synthetic generator did not expose it to.

7.5 Cause 4: side effects of per-window normalisation

The per-window centring of eq. (2.15) solves an obvious problem (the absolute dose is not informative) but creates a subtler one: when a window contains a sustained event such as `square` that covers most of its length, the window's mean is dragged *up* by the event itself. After centring, the event has *lost most of its amplitude*; what remains is the difference between the event level and a window mean that is itself elevated.

On the synthetic test set this is harmless because the events occupy a known fraction of the window, and the network has learned to expect a specific post-centring amplitude. On the real signal a long contamination plateau is essentially invisible after the same centring; only its edges survive, and they may be classified as `fast_ascend` / `fast_descend` but the plateau itself is not detected.

A second side-effect is that the per-window centring removes *any* information about absolute dose level. Two recordings of identical shape, one at 100 counts and one at 1 000 counts, are completely indistinguishable to the network. This is fine in principle but it means we cannot fall back on absolute-threshold reasoning at deployment time.

7.6 Cause 5: no "background-plus-glitch" samples in training

The training set is binary in spirit: a window either contains a clean templated event *or* it is pure synthetic background. Real recordings contain a much richer family of intermediate signals — a mild sensor wobble, an outlier sample, a slowly-correlated noise burst — that are neither a clean event nor pure stationary noise. The network does not have a class for "unknown signal that is not normal", so on any such region it is forced to pick one of the seven labels, and (because of cause 1) the most likely pick is one of the anomaly classes.

The CNN+Attention model's degenerate fixation on `M` (fig. 7.1, bottom panel) is the strongest illustration: the `M` template is the most "noisy-looking" of the six shapes (it has two peaks separated by a dip and a lot of internal texture), so on any input the network does not recognise, the attention layer settles on the class that "looks the most like something irregular".

7.7 Putting the causes together

The five effects compound:

1. The deployment prior is the opposite of the training prior ($\times 200$ mis-calibration in the Bayes-optimal posterior).
2. Class weights at training time make this worse by another factor of ~ 2.7.
3. Real anomalies frequently fall outside the duration band the templates cover.
4. Long events disappear after per-window centring.
5. There is no "unknown but not normal" class to fall back on, so the network is forced to choose an anomaly class even when nothing is happening.

The net result is a high false-positive rate (continuous stream of predicted events on quiet stretches), highly variable behaviour across architectures, and complete loss of operational utility.

7.8 Why each architecture fails in its own way

The same five-cause story does not predict why each model's failure looks different. The architectures' different inductive biases amplify the shift differently:

- **1D-CNN with BatchNorm.** BatchNorm running statistics, accumulated during training, are now applied to real data that does not match them. The resulting normalised activations are wildly different from anything the network saw at training, and the classifier head can

no longer commit to a single class — it oscillates because every class has roughly the same plausibility.

- **ResNet with GroupNorm.** GroupNorm is per-window, so normalisation behaves identically at training and deployment. The activations are valid, the features are stable — but the classifier head is still mis-calibrated by the prior shift, so it coherently and confidently picks the wrong class. This is, in some sense, the most dangerous failure mode: a confident wrong answer.
- **CNN+Attention.** The attention layer aggregates the entire window into a single CLS token. When the window is out-of-distribution, the CLS aggregation tends to converge to a single mode (the most-textured class, `M`) regardless of the input. This is consistent with the way Transformer encoders collapse on out-of-distribution inputs.

7.9 Remediation: a concrete punch-list for the next iteration

The diagnoses above translate directly into actionable changes.

R1. Remove class weights, or recalibrate at deployment. The simplest and most impactful change is to either (i) train without class weights, (ii) apply the Bayes prior correction of eq. (7.1) at deployment time, or (iii) calibrate the softmax temperature on a small held-out chunk of real, mostly-normal data.

R2. Heavily over-represent `Normal` at training. Re-run the data generator with one or two orders of magnitude fewer anomalies per million samples. Currently the synthetic data has ~ 7 events per 10 000 samples; bringing this down to ~ 0.5 would match the real prior much more closely.

R3. Add a "background-with-glitch" template family. Introduce synthetic templates for the kinds of out-of-distribution signals one would expect from a real sensor: single-sample `spike`, $1-2\%$ slow drift, sensor `dropout`, etc. This will give the network somewhere to put its confidence when the input is not normal but does not match any radiological template either.

R4. Drop per-window centring, or only centre by a running mean. Replace the window-by-window centring with a moving-average baseline estimated over a much wider window than $W = 512$ (e.g. $w = 5000$). This preserves the amplitude of sustained events while removing the slow seasonal drift.

R5. Expose the network to wider duration ranges. Add at least one template with very short duration (~ 5 samples) and one with very long duration (~ 2000 samples), or sweep the generator's $[200, 800]$ duration range to $[10, 5000]$ during dataset generation.

R6. Adopt a hierarchical two-stage pipeline. A first model decides *whether* the window is anomalous (a binary detector trained on a deployment-realistic prior); a second model is called only when the first one fires, to classify the anomaly into one of six classes. This decouples the prior-shift problem from the shape-classification problem, and the second stage can be trained on the current densely-anomalous synthetic data.

R7. Calibrate confidence with temperature scaling on real data. A small validation set of labelled real anomalies — even just the half-dozen plausible events visible across the four months of 2015 data — is enough to fit a single temperature parameter that flattens the network's softmax output. Doing this before R1 will make the rest of the pipeline far more interpretable.

The next iteration of this project will start with R1 and R2, which together address the prior-shift problem (the most consequential of the five causes), and then move to R3 and R4. The robustness study of chapter 6 suggests that the ResNet 1D architecture is the right vehicle for these changes: it has the best out-of-distribution behaviour of the three, and its parameter count is comfortably small enough to retrain frequently.

Chapter 8

Conclusion and future work

8.1 Summary of contributions

This project tackled the problem of detecting and classifying radiological anomalies in noisy gamma-dose time series. Starting from four months of real 2015 recordings, we built a complete pipeline from data to deployment-ready models:

1. A **noise model** was fitted on the real signal — a high-frequency Gaussian component on top of an Ornstein–Uhlenbeck baseline drift — and its parameters were extracted automatically by a calibration script.
2. Six **anomaly templates** were designed jointly with the supervisor as a structured CSV format (keypoints, allowed deformation quadrants, interpolation modes), making it straightforward to add new shapes in the future.
3. A scalable **synthetic dataset generator** produces a ten-million-point labelled signal in constant memory, mixing the noise model with random deformations of the six templates.
4. Three **neural network architectures** were implemented in PyTorch with a shared interface, ranging from a compact 23 000-parameter 1D-CNN to a 110 000-parameter hybrid CNN+Transformer model with multi-head self-attention.
5. A **sliding-window inference engine** and an **event-level evaluation protocol** bridge the gap between sample-level classification accuracy and the operational metric that matters (do we detect each anomaly event, and do we get its class right?).
6. A **robustness study** sweeps background noise and shape deformation independently and reveals the gracious-degradation properties of each architecture.
7. A **real-data deployment trial** on the original 2015 recordings produced an honest negative result — the models do not work well out of the box — followed by a careful diagnostic that identifies five distinct causes of the failure and a punch-list of seven concrete remediations.

8.2 Architecture recommendation

On the synthetic benchmark the ResNet 1D is the clear winner: it achieves a perfect 100% event-level macro F1 on the test segment, degrades the most gracefully under both noise and deformation stress, and remains compact enough ($\sim$ 75 000 parameters) to retrain in under ten minutes on a laptop GPU. The CNN+Attention model is competitive on synthetic data but exhibits a more severe out-of-distribution collapse on real data; we keep it in the pipeline mainly

because it offers the most promising path to longer-window analyses (section 8.4.2). The baseline 1D-CNN is the fastest to train but its catastrophic drop under high noise makes it unfit for a beacon that must run across seasons.

If a single architecture must be picked for the next iteration, the recommendation is **ResNet 1D**, possibly with a binary detection stage in front of it (section 7.9, R6).

8.3 Limitations

Several known limitations of the current pipeline are worth recording explicitly.

- **Distribution shift.** As detailed in chapter 7, the synthetic training distribution does not match the deployment distribution along several dimensions (class prior, anomaly duration, baseline drift, intermediate signals). The system as it stands cannot be deployed on raw beacon data without first addressing these mismatches.
- **No real ground truth.** The four months of real data have no expert labelling, so we have no quantitative measure of how the models do on real anomalies. The qualitative inspection in section 7.1 is the best we can do for now.
- **Single beacon.** The pipeline assumes a univariate time series. Real surveillance networks involve several beacons and several covariates (temperature, humidity, pressure). Exploiting these as additional input channels is left for future work.
- **Black-box decisions.** The trained models output a class label but not a justification. The project brief mentions Grad-CAM [7] as a candidate visualisation method; the architectures expose the relevant hooks (`last_conv_features`) but the Grad-CAM analysis itself was outside the time budget of this iteration.

8.4 Future work

The seven remediation steps of section 7.9 are the priority list for the next iteration. Beyond them, three longer-term directions are worth flagging.

8.4.1 Self-supervised pre-training on the real signal

A natural way to bridge the synthetic–real gap is to pre-train the convolutional stem on the four (and ideally more) months of real, unlabelled data, using a self-supervised objective such as masked-segment reconstruction. The stem then learns features adapted to the real signal statistics; only the classifier head needs to be fine-tuned on the synthetic dataset. This approach has been very successful in audio and biomedical time-series and is directly applicable here.

8.4.2 Longer windows and stronger attention

The CNN+Attention model did not benefit from its long-range mixing on 512-sample windows because the templates rarely extend past ~ 800 samples. With longer windows ($W = 4096$ or more) the attention model could in principle disambiguate close events (`M` versus two consecutive `bells`), reason about context (a slow ramp during which a sharp spike occurs), and detect multi-event episodes. Scaling the attention up requires the usual care (longer sinusoidal encoding, more heads or layers, possibly linear-complexity attention variants) but the architecture is already in place.

8.4.3 Explainability with Grad-CAM and attention visualisation

The two convolutional architectures expose their last convolutional layer as `last_conv_features`, ready to be hooked by a Grad-CAM implementation. For the attention model, the attention matrices themselves are interpretable saliency maps over the token sequence. Adding both to the inference scripts would let the supervisor visually verify that the model is paying attention to the right part of an event when it predicts a class, which is the kind of explainability the project brief explicitly requested.

8.4.4 Multi-beacon and multi-modal extensions

A practical surveillance system never works with a single signal in isolation; pressure, temperature, humidity and the gamma counts of neighbouring beacons are all informative. Extending the input to a small number of channels is mechanical (the convolutions are already multi-channel by design); the harder problem is collecting a labelled multi-modal dataset, which the synthetic generator could be extended to produce.

8.5 Final words

The project ends in an honest mixed state. On the benchmark we designed for ourselves the system works very well — all three architectures cross the 90% event-level F1 line and the residual network is essentially perfect. On the real data it does not, and the gap between the two is dominated by a small number of identifiable mismatches between the synthetic training distribution and the deployment distribution. Each of those mismatches has a known remediation; the next iteration of the project will apply them and re-run the same evaluation pipeline that is already in place. Many of the pieces required for the next round — the data generator, the event-level evaluator, the robustness study, the three architectures — are reusable as-is.

The single most useful lesson of this iteration is therefore not any particular F1 number but a methodological one: *do not benchmark a deep model on the same kind of data it was trained on, and call it done.* The first contact with real data is what revealed where the pipeline really stands, and the diagnostic infrastructure we built to make that contact diagnosable is, in the long run, the most valuable output of the project.

Bibliography

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] Z. Hu and L. Chen. Eeg-based emotion recognition using convolutional recurrent neural network with multi-head self attention. *Applied Sciences*, 12(21), 2022.

[5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015.

[6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[7] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[9] Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision (ECCV)*, 2018.

Appendix A

Appendices

A.1 Anomaly template CSV format

Each anomaly is encoded as a CSV file in `data-gen/anomalies/`. The columns are described below.

Column	Meaning
`time`	Normalised time of the keypoint, in $[0, 1]$.
`value`	Normalised amplitude of the keypoint, in $[0, 1]$.
`t_minus`, `t_plus`	Boolean flags allowing the keypoint to jitter in the negative or positive time direction.
`v_minus`, `v_plus`	Same, for the amplitude axis.
`interp`	Interpolation mode for the segment that *starts* at this keypoint: `linear` (straight line), `exp_up` (concave-up exponential), `exp_down` (concave-down exponential), `bell` (smoothed half-cosine).

Table A.1. Schema of the anomaly-template CSV files.

Example: `bell.csv`.

```
time,value,t_minus,t_plus,v_minus,v_plus,interp
0.0,0.0,0,0,0,0,bell
0.5,1.0,1,1,1,1,bell
1.0,0.0,0,0,0,0,linear
```

Listing A.1. Encoding of the `bell` template.

The first and last keypoints have all jitter flags off, so the template always starts and ends at $(0, 0)$ and $(1, 0)$. The middle keypoint can jitter in both time and amplitude; the two half-cosine segments make the curve smooth.

A.2 Hyperparameter reference

A.2.1 Data generator

Group	Parameter	Value
Size	Total points	10^7
	Chunk size	$5 \cdot 10^4$
Anomalies	Number of injections	uniform in $[5\,000, 8\,000]$
	Injection jitter	± 200 samples
Shapes	Amplitude (in σ)	uniform in $[2.5, 7.6]$
	Period (samples)	uniform in $[200, 800]$
	Deformation variance	uniform in $[0.02, 0.10]$
Background	Global mean μ	99.26
	HF noise std σ	2.53
	OU θ	$4 \cdot 10^{-6}$
	OU σ_{OU}	0.0373

Table A.2. Configuration of `generate_custom_dataset.py` used to produce the dataset for this report.

A.2.2 Training

Hyperparameter	Value
Window size W	512 samples
Sliding-window stride at training	32 samples
Sliding-window stride at inference	10 samples
Window labelling threshold	10% anomaly points
Per-window normalisation	subtract window mean, divide by global σ_{train}
σ_{train} used in this report	4.21
Train/val/test split	temporal 80/20 then random 80/20 on train
Optimizer	Adam, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$
Initial learning rate	10^{-3}
Batch size	128
Maximum epochs	35
LR scheduler	`ReduceLROnPlateau`(patience=2, factor=0.5)
Early stopping	patience 10, min delta $5 \cdot 10^{-3}$
Loss	weighted categorical cross-entropy
Class weights $\mathbf{w}$	$(0.46, 1.18, 1.27, 1.32, 1.24, 1.15, 1.31)$

Table A.3. Shared training configuration.

A.2.3 Inference and event extraction

Hyperparameter	Value
Sliding-window stride	10 samples
Inference batch size	256 windows
Smoothing window	30-sample rolling mean on the per-sample probability trace
Background-probability threshold for an event	$\hat{p}_0 < 0.3$
Tolerance gap between adjacent events	50 samples

Table A.4. Configuration of the inference engine and event extractor.

A.3 Reproducing the contents of this report

The repository layout is:

```
pole-anomaly/
  biblio/                  - the project brief and reference papers
  data-gen/                - real data + synthetic generator
    anomaly_generator.py     - template deformation + interpolation
    generate_custom_dataset.py - end-to-end dataset generator
    evaluate_real_data.py    - statistical characterisation of real data
    visualize_anomalies_templates.py
    interactive_labeling_helper.py
    anomalies/               - the six template CSV files
    data/                    - real recordings + generated CSV
    logs/                    - PNG/HTML inspection plots
  architectures/           - models, training, inference
    train.py                 - shared training pipeline (selects model)
    visualize_inference.py   - synthetic-test event-level evaluation
    inference_real_data.py   - real-data deployment trial
    robustness_test.py       - noise/deformation sweep
    inference_engine.py      - sliding-window + event matching
    model_selection.py       - model registry & CLI selector
    saved_models/            - 1d_cnn.py, resNet.py, cnn_attention.py
    logs/                    - per-model training/inference outputs
  documentation/           - this report
    main.tex, preamble.tex, sections/*.tex
    figures/build_figures.py - reproduces all PNGs in this report
```

Listing A.2. Top-level project structure.

The numbers and figures in this report can be reproduced from a clean checkout with the following command sequence:

```
# 1. Build the synthetic dataset (about 10 minutes; 600 MB on disk)
py data-gen/evaluate_real_data.py         # writes data/real_data_params.json
py data-gen/generate_custom_dataset.py    # writes
    data/synthetic_custom_dataset.csv

# 2. Make a copy of the dataset under architectures/data/ as
#    training_dataset.csv and validation_dataset.csv. (Same file twice is fine
#    if the user only wants to re-train; the temporal split inside train.py
#    keeps the test segment leakage-free.)

# 3. Train each architecture (~5-10 minutes each on an RTX-class GPU)
py architectures/train.py --model 1d_cnn
py architectures/train.py --model resnet
py architectures/train.py --model cnn_attention

# 4. Synthetic-test event-level evaluation
py architectures/visualize_inference.py --model 1d_cnn
py architectures/visualize_inference.py --model resnet
py architectures/visualize_inference.py --model cnn_attention

# 5. Robustness sweeps
py architectures/robustness_test.py --model 1d_cnn
py architectures/robustness_test.py --model resnet
py architectures/robustness_test.py --model cnn_attention

# 6. Real-data deployment trial
py architectures/inference_real_data.py --model 1d_cnn
py architectures/inference_real_data.py --model resnet
py architectures/inference_real_data.py --model cnn_attention

# 7. Rebuild the figures of this report
py documentation/figures/build_figures.py

# 8. Compile the report (with a local TeX install)
```

```
cd documentation && pdflatex main && pdflatex main
```

A.4 Software environment

All experiments were run on a single laptop with the following software stack:

- Windows 11, Python 3.13.
- PyTorch 2.12 (CUDA 12.8), running on an NVIDIA RTX 5060 Laptop GPU.
- NumPy, Pandas, Matplotlib, SciPy, scikit-learn for data handling, statistics and reporting.
- Plotly for the interactive HTML diagnostics shipped under `logs/`.

A pinned dependency list is included in the top-level `requirements.txt` of the repository.